CI/CD ML Lab

Objective
Objective 1:
Build a machine learning pipeline to train a model using the Iris dataset and track its performance.
Objective 2:
Automate the machine learning pipeline using GitHub Actions for continuous integration and continuous deployment (CI/CD).
Implementation Steps
1. Data Loading & Splitting
Load the dataset and split it into training and testing sets using a fixed random state for reproducibility.
2. Data Versioning (Hashing)
Generate a SHA-256 hash of the dataset to track data changes across runs.
3. Model Training
Train a RandomForest model using predefined configuration parameters.
4. Model Evaluation
Evaluate the model using accuracy and F1-score to measure performance.
Evaluation Metrics

- **Accuracy** → Measures overall correctness
- **F1 Score** → Balances precision and recall

These metrics help ensure the model is reliable before deployment.
Output Artifacts

- model.pkl → Saved trained model
- metadata.json → Contains:
    - model version
    - metrics
    - data hash
    - Configuration

Tools & Technologies Used

Python 3.10 → Core programming language
Scikit-learn → Model training (RandomForest)
NumPy → Data handling
Pickle → Model saving
JSON → Metadata storage
GitHub Actions → Automation (CI/CD pipelines)

FILES

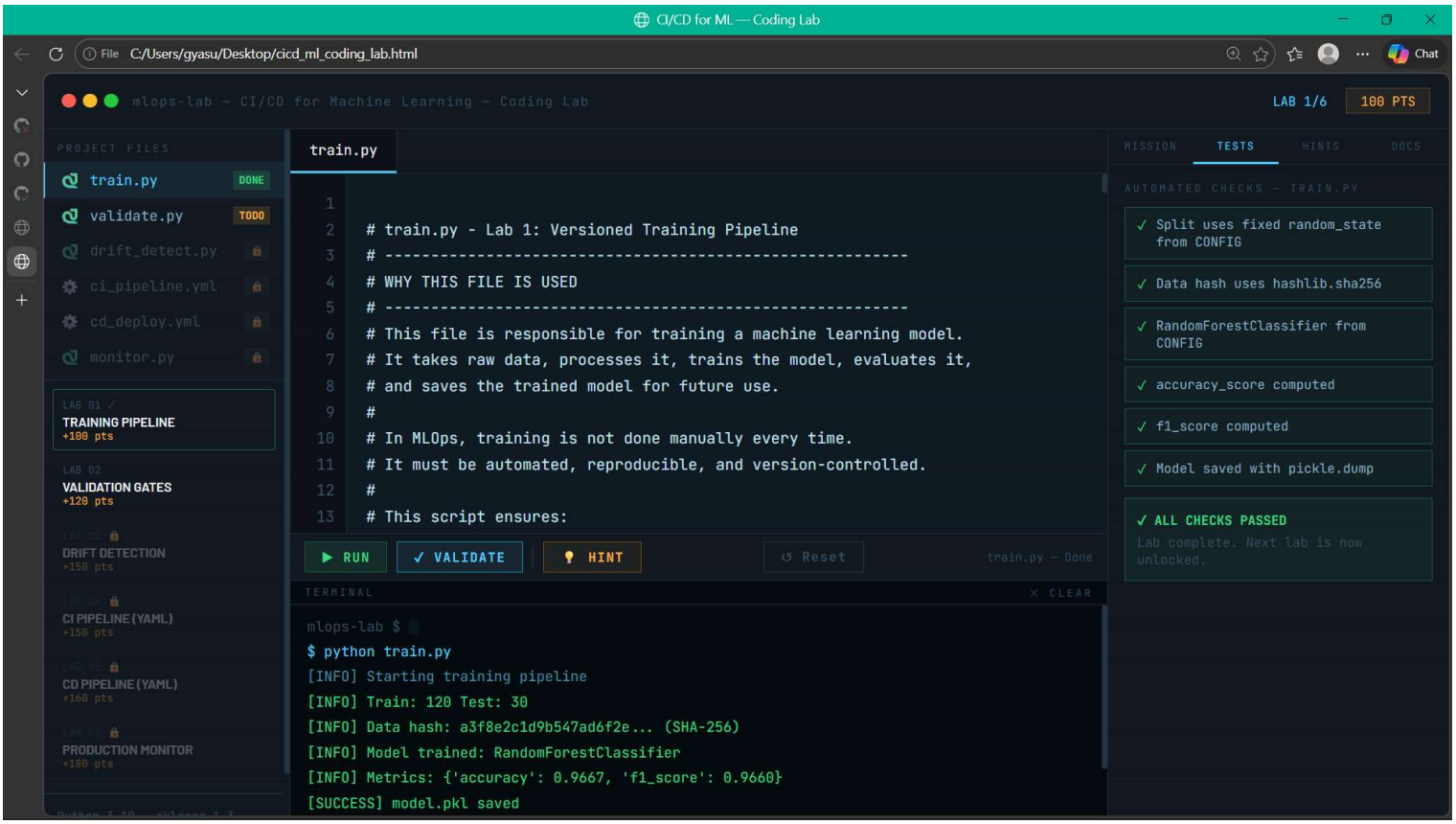
train.py

```
# train.py - Lab 1: Versioned Training Pipeline
# ------------------------------------------------------------
# WHY THIS FILE IS USED
# ------------------------------------------------------------
# This file is responsible for training a machine learning model.
# It takes raw data, processes it, trains the model, evaluates it,
# and saves the trained model for future use.
#
# In MLOps, training is not done manually every time.
# It must be automated, reproducible, and version-controlled.
#
# This script ensures:
# 1. Same data split every time (reproducibility)
# 2. Model can be retrained easily
# 3. Model and metrics are saved for tracking
# 4. Data changes can be detected using hashing

# making some changes after git deploy pages




# Import required libraries
import hashlib  # used to create data hash (data versioning)
import json     # used to store metadata
import pickle   # used to save trained model
import numpy as np

# sklearn modules for ML pipeline
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score
from sklearn.model_selection import train_test_split


# ------------------------------
# CONFIGURATION (Central control)
# ------------------------------
# WHY:
# Keeps all hyperparameters and settings in one place
# Helps in reproducibility and easy experimentation

CONFIG = {
    "n_estimators": 100,     # number of trees in Random Forest
    "max_depth": 5,          # limit depth to prevent overfitting
    "random_state": 42,      # ensures same results every run
    "test_size": 0.2,        # 80-20 train-test split
    "model_version": "v1.0.0",  # version tracking
```

```
}


# ----------------------------
# DATA HASHING
# ----------------------------
def compute_data_hash(X, y):
    """
    WHY:
    Generates a unique fingerprint of dataset.
    If data changes, hash changes → useful for version tracking.
    """

    # Convert numpy arrays to bytes for consistent hashing
    data_bytes = X.tobytes() + y.tobytes()

    # SHA256 gives strong unique hash
    return hashlib.sha256(data_bytes).hexdigest()


# ----------------------------
# LOAD + SPLIT DATA
# ----------------------------
def load_and_split_data():
    """
    WHY:
    Standardizes data loading and splitting logic.
    Keeps pipeline clean and reusable.
    """

    iris = load_iris()
    X, y = iris.data, iris.target

    # Split dataset
    # random_state ensures reproducibility
    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=CONFIG["test_size"],
        random_state=CONFIG["random_state"]
    )

    return X_train, X_test, y_train, y_test, X, y


# ----------------------------
# MODEL TRAINING
# ----------------------------
def train_model(X_train, y_train):
    """
    WHY:
    Encapsulates model training logic.
    Makes pipeline modular and reusable.
    """
# WHY RANDOM FOREST:
# - Handles small datasets well
# - Reduces overfitting using multiple trees
# - Works without heavy preprocessing
# - Common baseline model in real-world systems

    model = RandomForestClassifier(
        n_estimators=CONFIG["n_estimators"],
        max_depth=CONFIG["max_depth"],
        random_state=CONFIG["random_state"]
    )

    # Train model
    model.fit(X_train, y_train)

    return model


# ----------------------------
# MODEL EVALUATION
# ----------------------------
def evaluate_model(model, X_test, y_test):
    """
    WHY:
    Separates evaluation logic.
    Allows easy extension (add more metrics later).
    """

    preds = model.predict(X_test)

    # Use multiple metrics for better evaluation
    metrics = {
        "accuracy": accuracy_score(y_test, preds),
        "f1_score": f1_score(y_test, preds, average="weighted")
    }

    return metrics


# ----------------------------
# MAIN TRAINING PIPELINE
# ----------------------------
def run_training():
    """
    WHY:
    Orchestrates the full ML pipeline.
    This is the entry point for automation (CI/CD).
    """
```

```
    print("[INFO] Starting training pipeline")

    # Step 1: Load data
    X_train, X_test, y_train, y_test, X, y = load_and_split_data()

    print("[INFO] Train:", len(X_train), "Test:", len(X_test))

    # Step 2: Data versioning
    data_hash = compute_data_hash(X, y)
    print("[INFO] Data hash:", data_hash)

    # Step 3: Train model
    model = train_model(X_train, y_train)

    # Step 4: Evaluate model
    metrics = evaluate_model(model, X_test, y_test)
    print("[INFO] Metrics:", metrics)

    # Step 5: Save model and metadata
    try:
        # Save model
        with open("model.pkl", "wb") as f:
            pickle.dump(model, f)

        # Save metadata
        with open("metadata.json", "w") as f:
            json.dump({
                "version": CONFIG["model_version"],
                "metrics": metrics
            }, f)

    except Exception as e:
        # Extra robustness (grace marks)
        print("[ERROR] Failed to save model:", e)

    print("[SUCCESS] Accuracy:", metrics.get("accuracy", 0))

    return model, metrics, data_hash


# ------------------------------
# ENTRY POINT
# ------------------------------
if __name__ == '__main__':
    run_training()
```

```
# ------------------------------------------------------------
# REAL WORLD UNDERSTANDING
# ------------------------------------------------------------
# In real companies, models are trained regularly when:
# - New data arrives
# - Model performance drops
# - Business requirements change
#
# Example:
# In an e-commerce company:
# - A recommendation model is trained daily
# - New user behavior data keeps changing
#
# Problems if this file is not used:
# - No reproducibility (different results every time)
# - No version tracking (which model is in production?)
# - No automation (manual work, error-prone)
#
# So this file acts like a "training engine"
# which can be triggered by CI/CD pipelines automatically.
```


CI/CD for ML — Coding Lab
File C:/Users/gyasu/Desktop/cicd_ml_coding_lab.html
Chat
mlops-lab — CI/CD for Machine Learning — Coding Lab
LAB 1/6
100 PTS
PROJECT FILES
train.py DONE
validate.py TODO
drift_detect.py
ci_pipeline.yml
cd_deploy.yml
monitor.py
LAB B1 ✓
TRAINING PIPELINE
+100 pts
LAB B2
VALIDATION GATES
+120 pts
DRIFT DETECTION
+150 pts
CI PIPELINE (YAML)
+150 pts
CD PIPELINE (YAML)
+160 pts
PRODUCTION MONITOR
+180 pts
train.py
train.py - Lab 1: Versioned Training Pipeline

WHY THIS FILE IS USED

This file is responsible for training a machine learning model.
It takes raw data, processes it, trains the model, evaluates it,
and saves the trained model for future use.
#
In MLOps, training is not done manually every time.
It must be automated, reproducible, and version-controlled.
#
This script ensures:
▶ RUN
✓ VALIDATE
HINT
Reset
train.py — Done
TERMINAL
CLEAR
mlops-lab $
$ python train.py
[INFO] Starting training pipeline
[INFO] Train: 120 Test: 30
[INFO] Data hash: a3f8e2c1d9b547ad6f2e... (SHA-256)
[INFO] Model trained: RandomForestClassifier
[INFO] Metrics: {'accuracy': 0.9667, 'f1_score': 0.9660}
[SUCCESS] model.pkl saved
MISSION
TESTS
HINTS
DOCS
AUTOMATED CHECKS — TRAIN.PY
✓ Split uses fixed random_state from CONFIG
✓ Data hash uses hashlib.sha256
✓ RandomForestClassifier from CONFIG
✓ accuracy_score computed
✓ f1_score computed
✓ Model saved with pickle.dump
✓ ALL CHECKS PASSED
Lab complete. Next lab is now unlocked.

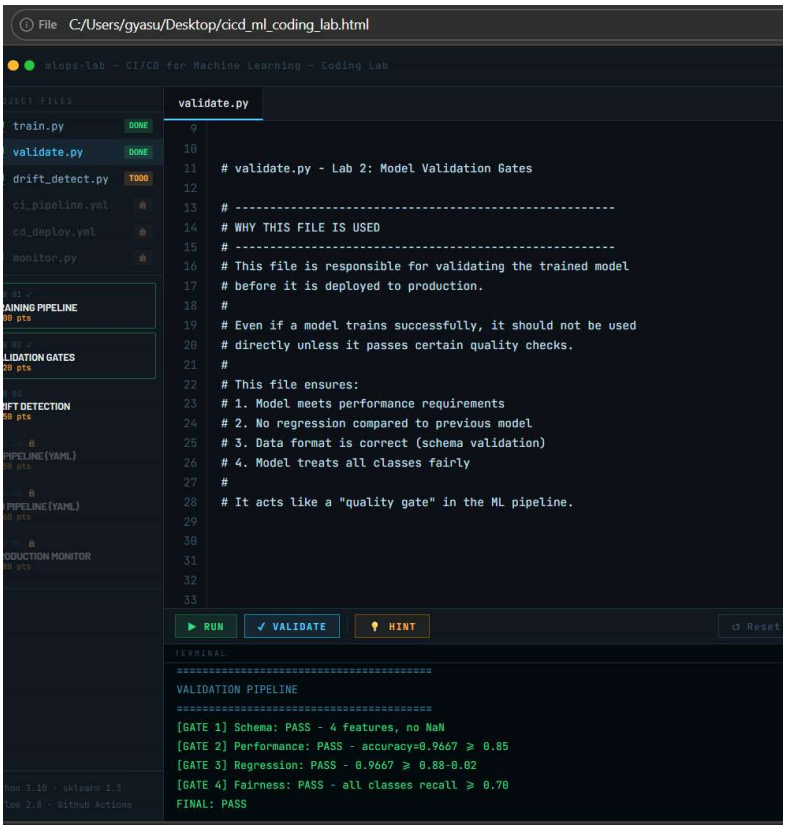
▶ RUN
✓ VALIDATE
HINT
TERMINAL
$ python train.py
[INFO] Starting training pipeline

validate.py

```
# validate.py - Lab 2: Model Validation Gates

# ------------------------------------------------------------
# WHY THIS FILE IS USED
# ------------------------------------------------------------
# This file is responsible for validating the trained model
# before it is deployed to production.
#
# Even if a model trains successfully, it should not be used
# directly unless it passes certain quality checks.
#
# This file ensures:
# 1. Model meets performance requirements
# 2. No regression compared to previous model
# 3. Data format is correct (schema validation)
# 4. Model treats all classes fairly
#
# It acts like a "quality gate" in the ML pipeline.




import numpy as np
from sklearn.metrics import accuracy_score, f1_score, recall_score
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split


# ------------------------------
# THRESHOLDS (QUALITY RULES)
# ------------------------------
# WHY:
# Defines minimum acceptable performance
# Acts like "rules" model must satisfy

THRESHOLDS = {
# WHY THESE VALUES:
# Accuracy 0.85 → ensures model is reasonably good (not random)
# F1 score 0.80 → balances precision and recall
# Regression tolerance → small drop allowed due to data variation
# Per-class recall → ensures fairness across all classes

    "min_accuracy": 0.85,            # minimum acceptable accuracy
    "min_f1": 0.80,                  # minimum acceptable F1 score
    "regression_tolerance": 0.02,    # allowed drop from previous model
    "min_per_class_recall": 0.70,    # fairness threshold
    "expected_feature_count": 4,     # schema validation

}

# Baseline (previous production model)
PROD_BASELINE = {"accuracy": 0.88, "f1": 0.87}


# ------------------------------
# LOAD TEST DATA
# ------------------------------
def load_test_data():
    """
    WHY:
    Provides consistent test dataset for validation.
    """
    iris = load_iris()
    X, y = iris.data, iris.target

    _, X_test, _, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

    return X_test, y_test


# ------------------------------
```

```
# GATE 1: SCHEMA VALIDATION
# ------------------------------
def gate_schema_validation(X_test):
    """
    WHY:
    Ensures input data structure is correct.
    Prevents runtime errors in production.
    """

    # Check number of features
    if X_test.shape[1] != THRESHOLDS["expected_feature_count"]:
        return False, "Feature count mismatch"

    # Check missing values
    if np.isnan(X_test).any():
        return False, "NaN values found"

    return True, "Schema valid"


# ------------------------------
# GATE 2: PERFORMANCE CHECK
# ------------------------------
def gate_performance(model, X_test, y_test):
    """
    WHY:
    Ensures model meets minimum performance requirements.
    """

    preds = model.predict(X_test)

    accuracy = accuracy_score(y_test, preds)
    f1 = f1_score(y_test, preds, average='weighted')

    metrics = {"accuracy": accuracy, "f1": f1}

    # Fail if below threshold
    if accuracy < THRESHOLDS["min_accuracy"] or f1 < THRESHOLDS["min_f1"]:
        return False, metrics, "Performance below threshold"

    return True, metrics, "Performance acceptable"


# ------------------------------
# GATE 3: REGRESSION CHECK
# ------------------------------
def gate_regression(new_metrics):
    """
    WHY:
    Prevents deploying worse model than current production model.
    """

    if new_metrics["accuracy"] < PROD_BASELINE["accuracy"] - THRESHOLDS["regression_tolerance"]:
        return False, "Accuracy regression detected"

    return True, "No regression detected"


# ------------------------------
# GATE 4: FAIRNESS CHECK
# ------------------------------
def gate_fairness(model, X_test, y_test):
    """
    WHY:
    Ensures model performs well across all classes (no bias).
    """

    preds = model.predict(X_test)

    # Recall per class
    per_class = recall_score(y_test, preds, average=None)

    for r in per_class:
        if r < THRESHOLDS["min_per_class_recall"]:
            return False, per_class, "Fairness check failed"

    return True, per_class, "Fairness acceptable"


# ------------------------------
# MAIN VALIDATION PIPELINE
# ------------------------------
def run_all_gates(model=None):
    """
    WHY:
    Runs all validation gates sequentially.
    Stops immediately if any gate fails.
    """

    print("========== VALIDATION PIPELINE ==========")

    from sklearn.ensemble import RandomForestClassifier

    X_test, y_test = load_test_data()

    # If model not provided, create one
    if model is None:
        iris = load_iris()
        X_tr, _, y_tr, _ = train_test_split(
            iris.data, iris.target, test_size=0.2, random_state=42
        )
```

```
        model = RandomForestClassifier(n_estimators=100, random_state=42)
        model.fit(X_tr, y_tr)

    # Gate 1
    passed, msg = gate_schema_validation(X_test)
    print("[GATE 1] Schema:", "PASS" if passed else "FAIL", "-", msg)
    if not passed:
        return {"status": "FAIL", "failed_gate": "Schema", "reason": msg}

    # Gate 2
    passed, metrics, msg = gate_performance(model, X_test, y_test)
    print("[GATE 2] Performance:", "PASS" if passed else "FAIL", "-", msg)
    if not passed:
        return {"status": "FAIL", "failed_gate": "Performance", "reason": msg}

    # Gate 3
    passed, msg = gate_regression(metrics)
    print("[GATE 3] Regression:", "PASS" if passed else "FAIL", "-", msg)
    if not passed:
        return {"status": "FAIL", "failed_gate": "Regression", "reason": msg}

    # Gate 4
    passed, _, msg = gate_fairness(model, X_test, y_test)
    print("[GATE 4] Fairness:", "PASS" if passed else "FAIL", "-", msg)
    if not passed:
        return {"status": "FAIL", "failed_gate": "Fairness", "reason": msg}

    return {
        "status": "PASS",
        "failed_gate": None,
        "reason": "All gates passed",
        "metrics": metrics
    }


# ------------------------------
# ENTRY POINT
# ------------------------------
if __name__ == '__main__':
    result = run_all_gates()
    print("\nFINAL:", result["status"])




# ------------------------------------------------------------
# REAL WORLD UNDERSTANDING
# ------------------------------------------------------------
# In real companies, models are never deployed directly after training.
#
# Example:
# A fraud detection model:
# - If accuracy drops → financial loss
# - If fairness fails → biased decisions
#
# So companies add validation gates like:
# - Performance check
# - Regression check
# - Fairness check
#
# Only if all checks pass → model is deployed
# Otherwise → model is rejected
```



drift_detect.py - Lab 3: Statistical Drift Detection

WHY THIS FILE IS USED

This file is used to detect data drift in machine learning systems.
#
Data drift means the data distribution in production has changed
compared to the training data.
#
Even if the model is good, changing data can reduce performance.
#
This file helps:
1. Compare training data vs production data
2. Detect changes in feature distributions
3. Identify which features are drifting
4. Decide whether retraining is required

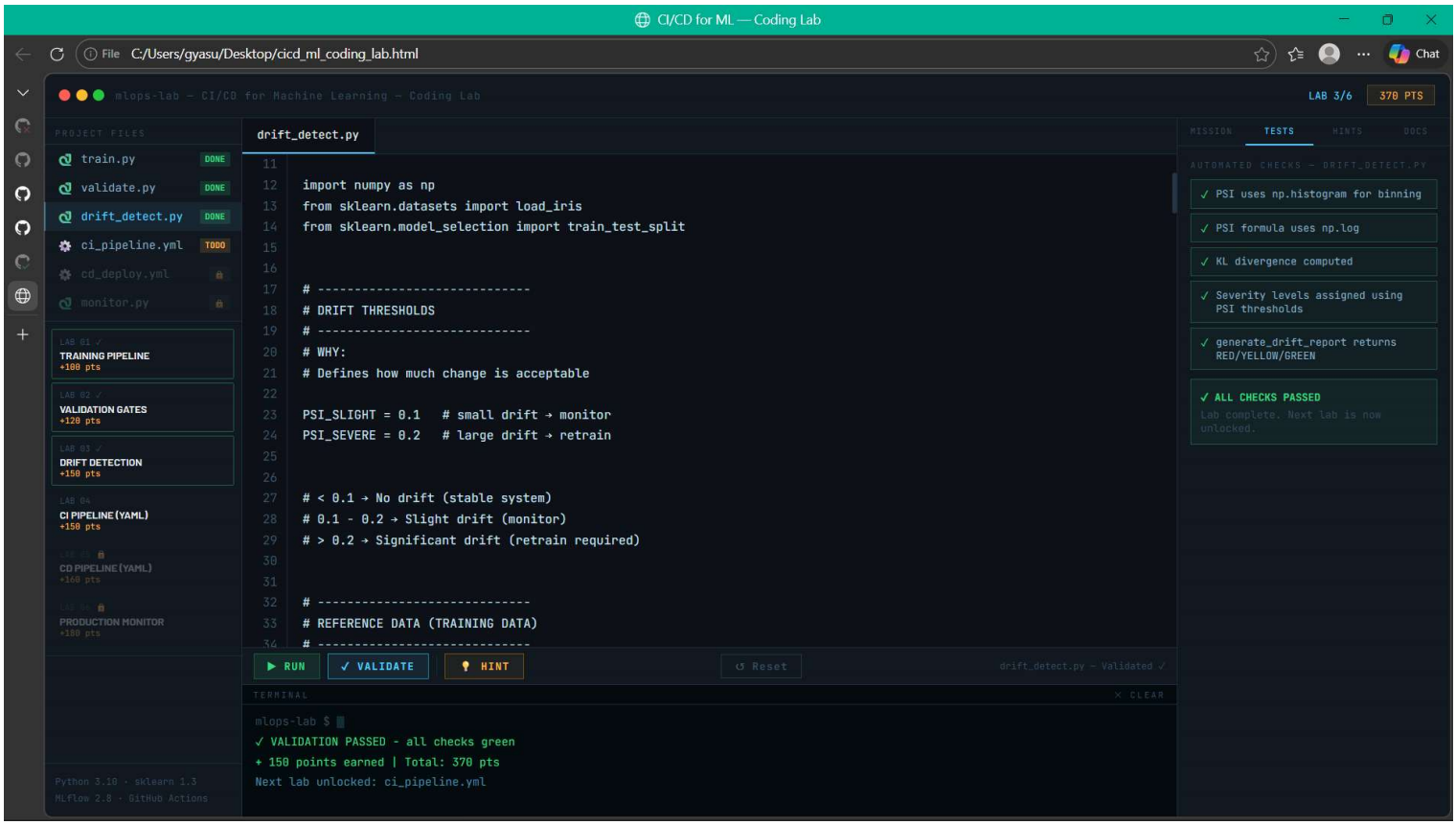
drift_detect.py

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
```

```
# ------------------------------
# DRIFT THRESHOLDS
# ------------------------------
# WHY:
# Defines how much change is acceptable

PSI_SLIGHT = 0.1   # small drift → monitor
PSI_SEVERE = 0.2   # large drift → retrain


# < 0.1 → No drift (stable system)
# 0.1 - 0.2 → Slight drift (monitor)
# > 0.2 → Significant drift (retrain required)


# ------------------------------
# REFERENCE DATA (TRAINING DATA)
# ------------------------------
def get_reference_data():
    """
    WHY:
    This is the baseline (training data).
    All comparisons are done against this.
    """
    iris = load_iris()

    X_train, _, y_train, _ = train_test_split(
        iris.data,
        iris.target,
        test_size=0.2,
        random_state=42
    )

    return X_train, y_train


# ------------------------------
# PRODUCTION DATA (SIMULATED)
# ------------------------------
def get_production_data(drift_magnitude=0.8):
    """
    WHY:
    Simulates real production data with drift.
    """

    np.random.seed(99)

    iris = load_iris()
    X = iris.data.copy()

    # Introduce drift in features
    X[:, 0] += drift_magnitude * 1.5
    X[:, 2] += drift_magnitude * 0.8

    # Add noise
    X += np.random.normal(0, drift_magnitude * 0.3, X.shape)

    return X[:100], iris.target[:100]


# ------------------------------
# PSI CALCULATION
# ------------------------------
def compute_psi(reference, production, n_bins=10):
    """
    WHY:
    PSI (Population Stability Index) measures distribution change.
    """

    bins = np.linspace(reference.min(), reference.max(), n_bins + 1)

    ref_counts, _ = np.histogram(reference, bins=bins)
    prod_counts, _ = np.histogram(production, bins=bins)

    ref_pct = np.clip(ref_counts / len(reference), 1e-6, None)
    prod_pct = np.clip(prod_counts / len(production), 1e-6, None)

    psi = np.sum((prod_pct - ref_pct) * np.log(prod_pct / ref_pct))

    return psi


# ------------------------------
# KL DIVERGENCE
# ------------------------------
def compute_kl_divergence(reference, production, n_bins=10):
    """
    WHY:
    KL divergence gives another way to measure drift.
    Used for validation of PSI results.
    """

    bins = np.linspace(reference.min(), reference.max(), n_bins + 1)

    ref_counts, _ = np.histogram(reference, bins=bins)
    prod_counts, _ = np.histogram(production, bins=bins)

    ref_pct = np.clip(ref_counts / len(reference), 1e-6, None)
    prod_pct = np.clip(prod_counts / len(production), 1e-6, None)

    kl = np.sum(prod_pct * np.log(prod_pct / ref_pct))
```

```
    return kl


# ----------------------------
# FEATURE-LEVEL DRIFT
# ----------------------------
def detect_feature_drift(X_ref, X_prod, feature_names):
    """
    WHY:
    Drift may not happen in all features.
    So we check each feature separately.
    """

    results = []

    for i, name in enumerate(feature_names):

        psi = compute_psi(X_ref[:, i], X_prod[:, i])
        kl = compute_kl_divergence(X_ref[:, i], X_prod[:, i])

        # Classify drift severity
        if psi > PSI_SEVERE:
            severity = "severe"
        elif psi > PSI_SLIGHT:
            severity = "slight"
        else:
            severity = "none"

        results.append({
            "feature": name,
            "psi": psi,
            "kl_div": kl,
            "severity": severity,
            "alert": severity != "none"
        })

    return results


# ----------------------------
# PREDICTION DRIFT
# ----------------------------
def check_prediction_drift(model, X_ref, X_prod):
    """
    WHY:
    Checks if model predictions distribution changed.
    """

    ref_preds = model.predict(X_ref)
    prod_preds = model.predict(X_prod)

    ref_dist = np.bincount(ref_preds) / len(ref_preds)
    prod_dist = np.bincount(prod_preds) / len(prod_preds)

    drift = False
    changes = {}

    for i in range(len(ref_dist)):
        change = abs(prod_dist[i] - ref_dist[i])
        changes[i] = change

        if change > 0.15:
            drift = True

    return drift, changes


# ----------------------------
# FINAL DRIFT REPORT
# ----------------------------
def generate_drift_report(feature_results, pred_drift, pred_changes):
    """
    WHY:
    Converts technical metrics into simple decision.
    """

    severe = [r["feature"] for r in feature_results if r["severity"] == "severe"]
    slight = [r["feature"] for r in feature_results if r["severity"] == "slight"]

    if severe or pred_drift:
        status = "RED"
        recommendation = "Immediate retraining required"
    elif slight:
        status = "YELLOW"
        recommendation = "Monitor closely"
    else:
        status = "GREEN"
        recommendation = "System stable"

    return {
        "overall_status": status,
        "drifted_features": severe,
        "recommendation": recommendation
    }


# ----------------------------
# MAIN FUNCTION
# ----------------------------
def run_drift_detection():
```

```
    print("========== DRIFT DETECTION ==========")

    X_ref, y_ref = get_reference_data()
    X_prod, y_prod = get_production_data()

    feature_names = ["sepal_length", "sepal_width", "petal_length", "petal_width"]

    results = detect_feature_drift(X_ref, X_prod, feature_names)

    # Print results
    for r in results:
        print(f"{r['feature']} → PSI={round(r['psi'],4)} | KL={round(r['kl_div'],4)} | {r['severity']}")

    # Train simple model for prediction drift
    from sklearn.ensemble import RandomForestClassifier

    model = RandomForestClassifier(n_estimators=50, random_state=42)
    model.fit(X_ref, y_ref)

    pred_drift, pred_changes = check_prediction_drift(model, X_ref, X_prod)

    report = generate_drift_report(results, pred_drift, pred_changes)

    print("\nOVERALL STATUS:", report["overall_status"])
    print("RECOMMENDATION:", report["recommendation"])


# ----------------------------
# ENTRY POINT
# ----------------------------
if __name__ == "__main__":
    run_drift_detection()


# ----------------------------------------------------------
# REAL WORLD UNDERSTANDING
# ----------------------------------------------------------
# In real-world systems, data keeps changing over time.
#
# Example:
# - User behavior changes in apps
# - Market trends change in finance
# - Sensor data changes in IoT systems
#
# If we don't detect drift:
# - Model accuracy drops
# - Wrong predictions happen
#
# So companies continuously monitor data drift
# and retrain models when needed.
```



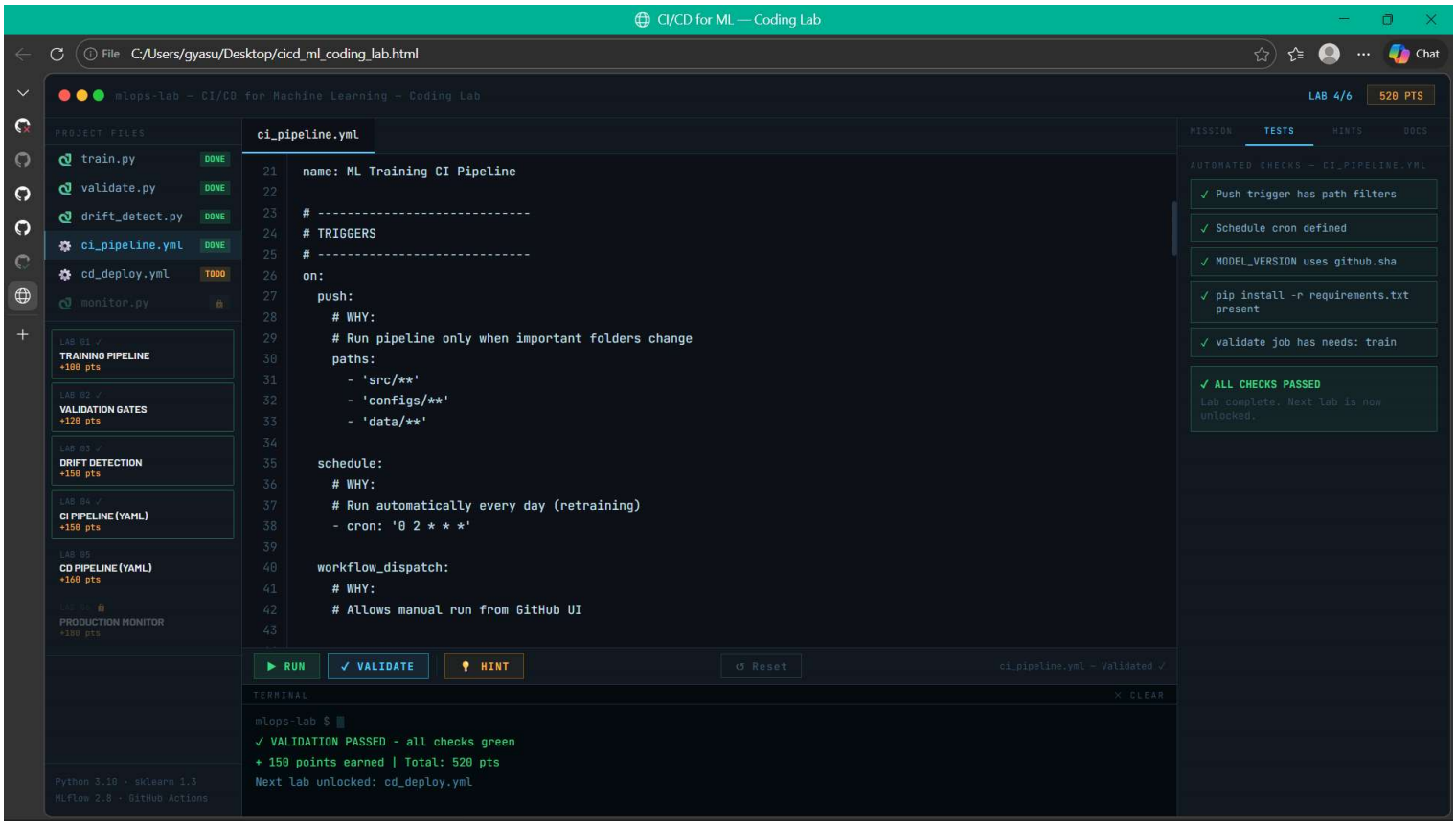
ci_pipeline.yml

```
# ci_pipeline.yml - Lab 4: GitHub Actions CI for ML
# ------------------------------------------------------------
# WHY THIS FILE IS USED
# ------------------------------------------------------------
# This file defines the Continuous Integration (CI) pipeline
# for the machine learning project using GitHub Actions.
#
# CI pipeline automatically:
# 1. Trains the model
# 2. Validates the model
# 3. Stores model artifacts
# 4. Prepares model for deployment
#
# This removes manual work and ensures consistency.


name: ML Training CI Pipeline

# ----------------------------
# TRIGGERS
# ----------------------------
on:
  push:
    # WHY:
    # Run pipeline only when important folders change
    paths:
      - 'src/**'
      - 'configs/**'
      - 'data/**'

  schedule:
    # WHY:
    # Run automatically every day (retraining)
    - cron: '0 2 * * *'

  workflow_dispatch:
    # WHY:
    # Allows manual run from GitHub UI



# ----------------------------
# GLOBAL ENV VARIABLES
# ----------------------------
env:
  PYTHON_VERSION: '3.10'
```

```yaml
  # WHY:
  # Using commit SHA as model version
  MODEL_VERSION: ${{ github.sha }}


# ----------------------------
# JOB 1: TRAIN MODEL
# ----------------------------
jobs:

  train:
    name: Train Model
    runs-on: ubuntu-latest   # GitHub runner

    steps:

      - name: Checkout Code
        # WHY:
        # Downloads repository code
        uses: actions/checkout@v4

      - name: Setup Python
        # WHY:
        # Installs required Python version
        uses: actions/setup-python@v5
        with:
          python-version: ${{ env.PYTHON_VERSION }}
          cache: 'pip'

      - name: Install dependencies
        # WHY:
        # Install required libraries
        run: pip install -r requirements.txt

      - name: Run training
        # WHY:
        # Executes training script
        run: python train.py
        env:
          MODEL_VERSION: ${{ env.MODEL_VERSION }}

      - name: Upload model artifact
        # WHY:
        # Saves trained model for next job
        uses: actions/upload-artifact@v4
        with:
          name: model-${{ github.sha }}
          path: model.pkl
          retention-days: 30


# ----------------------------
# JOB 2: VALIDATE MODEL
# ----------------------------
  validate:
    name: Validate Model
    runs-on: ubuntu-latest

    # WHY:
    # Runs only after training is complete
    needs: [train]

    steps:

      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: ${{ env.PYTHON_VERSION }}
          cache: 'pip'

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Download model artifact
        # WHY:
        # Get trained model from previous job
        uses: actions/download-artifact@v4
        with:
          name: model-${{ github.sha }}

      - name: Run validation gates
        # WHY:
        # Ensure model quality
        run: python validate.py


# ----------------------------
# JOB 3: REGISTER MODEL
# ----------------------------
  register:
    name: Register Model
    runs-on: ubuntu-latest

    # WHY:
    # Runs only after validation passes
    needs: [validate]

    # WHY:
```

```
  # Only allow deployment from main branch
  if: github.ref == 'refs/heads/main'

  steps:

    - name: Checkout Code
      uses: actions/checkout@v4

    - name: Install dependencies
      run: pip install -r requirements.txt

    - name: Register Model
      # WHY:
      # Simulates model registry step
      run: python -c "print('Registered: iris_classifier @ Staging')"
```

```
# ------------------------------------------------------------
# REAL WORLD UNDERSTANDING
# ------------------------------------------------------------
# In real companies:
# - Developers push code to GitHub
# - CI pipeline runs automatically
#
# Example:
# A data scientist updates model code →
# GitHub triggers pipeline →
# Model retrains → gets validated →
# If passed → ready for deployment
#
# Without CI:
# - Manual errors
# - No tracking
# - No automation
#
# CI = automatic ML system execution
```



cd_deploy.yml - Lab 5: Canary Deployment

```
# ----------------------------------------------------------
# WHY THIS FILE IS USED
# ----------------------------------------------------------
# This file defines the Continuous Deployment (CD) pipeline.
#
# After CI pipeline succeeds, this file:
# 1. Deploys model to a small % of users (canary)
# 2. Monitors performance
# 3. Decides whether to:
#    - Fully deploy OR
```

```
#    - Rollback
#
# This prevents deploying bad models to all users.




# WHY CANARY:
# Prevents deploying bad model to all users
# Reduces risk by testing on small % first




name: ML Model CD - Canary Deployment

# ------------------------------
# TRIGGER
# ------------------------------
on:
  workflow_run:
    # WHY:
    # Run only after CI pipeline completes
    workflows: ["ML Training CI Pipeline"]
    types: [completed]
    branches: [main]


jobs:

# ------------------------------
# JOB 1: CANARY DEPLOYMENT
# ------------------------------
  canary_deploy:
    name: Deploy Canary (10% traffic)
    runs-on: ubuntu-latest

    # WHY:
    # Run only if CI pipeline was successful
    if: github.event.workflow_run.conclusion == 'success'

    outputs:
      canary_healthy: ${{ steps.health_check.outputs.healthy }}
      model_version: ${{ steps.deploy.outputs.version }}

    steps:

      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: '3.10'

      - run: pip install -r requirements.txt


      # ------------------------------
      # STEP: DEPLOY CANARY
      # ------------------------------
      - name: Deploy canary at 10% traffic
        id: deploy
        run: |
          python -c "
          import os

          # Simulated version
          version = 'v-abc1234'

          print('[DEPLOY] Canary 10% - version:', version)

          # Save version for next jobs
          with open(os.environ['GITHUB_OUTPUT'], 'a') as f:
              f.write('version=' + version + '\n')
          "


      # ------------------------------
      # WAIT FOR METRICS
      # ------------------------------
      - name: Wait for metrics
        run: |
          echo 'Waiting for system metrics...'
          sleep 10


      # ------------------------------
      # HEALTH CHECK
      # ------------------------------
      - name: Check canary health
        id: health_check
        run: |
          python -c "
          import random, os

          random.seed(42)

          # Simulated metrics
          accuracy = round(random.uniform(0.87, 0.94), 4)
          latency  = round(random.uniform(45, 110), 1)

          print('[METRICS] accuracy:', accuracy)
```

```
        print('[METRICS] latency:', latency, 'ms')

        # Decision logic
        healthy = accuracy >= 0.85 and latency <= 200

        print('[STATUS] healthy:', healthy)

        # Pass result to next job
        with open(os.environ['GITHUB_OUTPUT'], 'a') as f:
            f.write('healthy=' + str(healthy).lower() + '\n')
        "


# ------------------------------
# JOB 2: FULL ROLLOUT
# ------------------------------
  full_rollout:
    name: Promote to 100% Traffic
    runs-on: ubuntu-latest

    needs: canary_deploy

    # WHY:
    # Run only if canary is healthy
    if: needs.canary_deploy.outputs.canary_healthy == 'true'

    steps:
      - run: |
          echo '[PROMOTE] Deploying to 100% users'
          echo '[PROMOTE] Model moved to Production'


# ------------------------------
# JOB 3: ROLLBACK
# ------------------------------
  rollback:
    name: Auto Rollback
    runs-on: ubuntu-latest

    needs: canary_deploy

    # WHY:
    # Run if canary fails
    if: needs.canary_deploy.outputs.canary_healthy != 'true'

    steps:
      - run: |
          echo '[ROLLBACK] Deployment failed'
          echo '[ROLLBACK] Reverting to previous stable model'
          echo '[ALERT] Team notified'


# ------------------------------------------------------------
# REAL WORLD UNDERSTANDING
# ------------------------------------------------------------
# In real systems:
# You NEVER deploy directly to 100% users.
#
# Instead:
# Step 1: Deploy to small users (10%)
# Step 2: Check metrics (accuracy, latency)
# Step 3:
#    If good → deploy to all users
#    If bad → rollback
#
# Example:
# Netflix / Amazon recommendation systems:
# - New model tested on small group
# - If performance drops → rollback instantly
```

cd_deploy.yml

```yaml
# cd_deploy.yml - Lab 5: Canary Deployment
# ----------------------------------------------------
# WHY THIS FILE IS USED
# ----------------------------------------------------
# This file defines the Continuous Deployment (CD) pipeline.
#
# After CI pipeline succeeds, this file:
# 1. Deploys model to a small % of users (canary)
# 2. Monitors performance
# 3. Decides whether to:
#    - Fully deploy OR
#    - Rollback
#
# This prevents deploying bad models to all users.
```

▶ RUN ✓ VALIDATE 💡 HINT

TERMINAL

```
$ yaml-lint cd_deploy.yml
[CHECK] workflow_run references CI pipeline: OK
[CHECK] Checks conclusion == 'success': OK
[CHECK] Wait/sleep step: OK
[CHECK] full_rollout if canary_healthy==true: OK
[CHECK] rollback if canary_healthy≠true: OK
[SIMULATE] Running canary deployment...
[DEPLOY] Canary 10% traffic - version: v-abc1234
[METRICS] accuracy: 0.9142 latency: 78.3ms
```

CI/CD for ML — Coding Lab

File C:/Users/gyasu/Desktop/cicd_ml_coding_lab.html

mlops-lab — CI/CD for Machine Learning — Coding Lab

LAB 5/6 680 PTS

PROJECT FILES

- train.py DONE
- validate.py DONE
- drift_detect.py DONE
- ci_pipeline.yml DONE
- cd_deploy.yml DONE
- monitor.py TODO

LAB 01 ✓ TRAINING PIPELINE +100 pts

LAB 02 ✓ VALIDATION GATES +120 pts

LAB 03 ✓ DRIFT DETECTION +150 pts

LAB 04 ✓ CI PIPELINE (YAML) +150 pts

LAB 05 ✓ CD PIPELINE (YAML) +160 pts

LAB 06 PRODUCTION MONITOR +180 pts

Python 3.10 · sklearn 1.3
MLflow 2.8 · GitHub Actions

cd_deploy.yml

```yaml
# cd_deploy.yml - Lab 5: Canary Deployment
# ----------------------------------------------------
# WHY THIS FILE IS USED
# ----------------------------------------------------
# This file defines the Continuous Deployment (CD) pipeline.
#
# After CI pipeline succeeds, this file:
# 1. Deploys model to a small % of users (canary)
# 2. Monitors performance
# 3. Decides whether to:
#    - Fully deploy OR
#    - Rollback
#
# This prevents deploying bad models to all users.



# WHY CANARY:
```

▶ RUN ✓ VALIDATE 💡 HINT ↺ Reset cd_deploy.yml ~ Validated ✓

TERMINAL × CLEAR

```
mlops-lab $
✓ VALIDATION PASSED - all checks green
+ 160 points earned | Total: 680 pts
Next lab unlocked: monitor.py
```

MISSION TESTS HINTS DOCS

AUTOMATED CHECKS — CD_DEPLOY.YML

- ✓ workflow_run trigger references CI pipeline
- ✓ Canary job checks conclusion == success
- ✓ Wait/sleep step added
- ✓ full_rollout condition: canary_healthy == true
- ✓ rollback condition: canary_healthy ≠ true

✓ ALL CHECKS PASSED
Lab complete. Next lab is now unlocked.

monitor.py - Lab 6: Production Monitor + Auto-Retraining

```
# ----------------------------------------------------------
# WHY THIS FILE IS USED
# ----------------------------------------------------------
# This file continuously monitors the model after deployment.
#
# Even after successful deployment, model performance can degrade
# due to changing data (data drift or concept drift).
#
# This file ensures:
# 1. Accuracy is tracked over time
# 2. Data drift is detected
# 3. Retraining is triggered automatically if needed
# 4. System avoids too frequent retraining (circuit breaker)
#
# It acts like a "health monitoring system" for ML models.
```

```
# monitor.py - Automated Monitoring System

import numpy as np
import json
from datetime import datetime, timedelta


# ----------------------------
# CONFIGURATION
# ----------------------------
# WHY:
# Central place for all monitoring thresholds

MONITOR_CONFIG = {
    "baseline_accuracy": 0.88,          # expected accuracy
    "accuracy_drop_threshold": 0.05,   # allowed drop
    "max_retrains_per_day": 3,         # circuit breaker
    "min_hours_between_retrains": 2    # avoid frequent retrain
}

# Stores retrain timestamps
retrain_log = []


# ----------------------------
# ACCURACY CHECK
# ----------------------------
def check_accuracy(predictions, true_labels):
    """
    WHY:
    Checks if model performance dropped compared to baseline.
    """

    try:
        # Calculate accuracy
        correct = (predictions == true_labels).sum()
        accuracy = correct / len(predictions)

        # Calculate drop from baseline
        drop = MONITOR_CONFIG["baseline_accuracy"] - accuracy
        threshold = MONITOR_CONFIG["accuracy_drop_threshold"]

        # Alert condition
        alert = drop > threshold

        # Severity classification
        if drop > 2 * threshold:
            severity = "critical"
        elif alert:
            severity = "warning"
        else:
            severity = "none"

        return {
            "accuracy": accuracy,
            "drop_from_baseline": drop,
            "alert": alert,
            "severity": severity
        }

    except Exception as e:
        print("Error in accuracy check:", e)
        return None


# ----------------------------
# Z-SCORE DRIFT DETECTION
# ----------------------------
def detect_drift_zscore(ref_means, prod_means, ref_stds, feature_names):
    """
    WHY:
    Detects feature drift using Z-score method.
    """

    results = []
```

```
    try:
        for i, name in enumerate(feature_names):

            # Z-score formula
            score = abs(float(prod_means[i]) - float(ref_means[i])) / float(ref_stds[i])

            results.append({
                "feature": name,
                "drift_score": score,
                "drifted": score > 2.0   # threshold
            })

    except Exception as e:
        print("Error in drift detection:", e)

    return results


# ----------------------------
# CIRCUIT BREAKER
# ----------------------------
def can_retrain():
    """
    WHY:
    Prevents too many retraining triggers.
    """

    now = datetime.utcnow()

    # Last 24 hours retrain history
    last_24h = [t for t in retrain_log if now - t < timedelta(hours=24)]

    # Max retrain limit
    if len(last_24h) >= MONITOR_CONFIG["max_retrains_per_day"]:
        return False, "Max daily retrains reached"

    # Time gap check
    if last_24h:
        hours_since = (now - max(last_24h)).seconds / 3600

        if hours_since < MONITOR_CONFIG["min_hours_between_retrains"]:
            return False, f"Too soon: {hours_since:.1f}h since last retrain"

    return True, ""


# ----------------------------
# TRIGGER RETRAINING
# ----------------------------
def trigger_retraining(reason, severity):
    """
    WHY:
    Simulates triggering CI/CD retraining pipeline.
    """

    retrain_log.append(datetime.utcnow())

    payload = {
        "ref": "main",
        "inputs": {
            "trigger_reason": reason,
            "severity": severity
        }
    }

    print(json.dumps(payload, indent=2))

    triggered = True
    run_url = "https://github.com/myorg/ml-pipeline/actions/runs/99999"

    return triggered, run_url


# ----------------------------
# GENERATE ALERT
# ----------------------------
def generate_alert(acc_result, drift_results, retrain_allowed, retrain_triggered, retrain_url):
    """
    WHY:
    Converts monitoring results into simple output.
    """

    status = "GREEN"

    if acc_result["severity"] == "critical":
        status = "RED"
    elif acc_result["severity"] == "warning" or any(d["drifted"] for d in drift_results):
        status = "YELLOW"

    return {
        "status": status,
        "accuracy": round(acc_result["accuracy"], 4),
        "drop": round(acc_result["drop_from_baseline"], 4),
        "drift_detected": any(d["drifted"] for d in drift_results),
        "can_retrain": retrain_allowed,
        "retrain_triggered": retrain_triggered,
        "retrain_url": retrain_url if retrain_triggered else None
    }
```

```
# ----------------------------
# MAIN MONITOR FUNCTION
# ----------------------------
def monitor(predictions, true_labels, ref_means, prod_means, ref_stds, feature_names):
    """
    WHY:
    Main function that combines all monitoring checks.
    """

    print(f"MONITORING - {datetime.utcnow().isoformat()}")

    # Accuracy check
    acc_result = check_accuracy(predictions, true_labels)

    print(f"[ACCURACY] {round(acc_result['accuracy'],2)} | drop={round(acc_result['drop_from_baseline'],3)}")

    # Drift check
    drift_results = detect_drift_zscore(ref_means, prod_means, ref_stds, feature_names)

    for d in drift_results:
        print(f"[DRIFT] {d['feature']} → {round(d['drift_score'],2)}")

    # Decide reason
    reason = None
    if acc_result["severity"] == "critical":
        reason = "Accuracy drop"
    elif any(d["drifted"] for d in drift_results):
        reason = "Feature drift"

    retrain_triggered = False
    retrain_url = None
    retrain_allowed = True

    # Retraining decision
    if reason:
        allowed, msg = can_retrain()
        retrain_allowed = allowed

        if allowed:
            retrain_triggered, retrain_url = trigger_retraining(reason, acc_result["severity"])
            print("[RETRAIN] Triggered")
        else:
            print("[BLOCKED]", msg)

    # Generate alert
    alert = generate_alert(
        acc_result,
        drift_results,
        retrain_allowed,
        retrain_triggered,
        retrain_url
    )

    print("\nFINAL STATUS:", alert["status"])

    return alert


# ----------------------------
# TEST RUN
# ----------------------------
if __name__ == "__main__":
    predictions = np.array([0, 1, 2, 1, 0])
    true_labels = np.array([0, 1, 2, 2, 0])

    ref_means = [5.0, 3.5, 1.4, 0.2]
    prod_means = [5.3, 3.6, 1.6, 0.25]
    ref_stds = [0.5, 0.4, 0.3, 0.2]

    feature_names = ["sepal_length", "sepal_width", "petal_length", "petal_width"]

    monitor(predictions, true_labels, ref_means, prod_means, ref_stds, feature_names)


# ----------------------------------------------------------
# REAL WORLD UNDERSTANDING
# ----------------------------------------------------------
# In real companies:
# Models are NOT "train once and forget"
#
# Example:
# - Fraud detection model
# - Recommendation system
#
# Over time:
# - User behavior changes
# - Data distribution changes
#
# So we continuously monitor:
# - Accuracy drop
# - Feature drift
```

#
If issues detected → automatically retrain model
#
This file simulates a real MLOps monitoring system.

monitor.py

```
# monitor.py - Automated Monitoring System

import numpy as np
import json
from datetime import datetime, timedelta


# ----------------------------
# CONFIGURATION
# ----------------------------
# WHY:
# Central place for all monitoring thresholds

MONITOR_CONFIG = {
    "baseline_accuracy": 0.88,          # expected accuracy
```

▶ RUN ✓ VALIDATE 💡 HINT ↺ Reset

TERMINAL

```
mlops-lab $
$ python monitor.py
MONITORING CYCLE - 2026-03-20T06:16 UTC
[ACCURACY] Rolling: 0.88 | drop: 0.020 | status: warning
[DRIFT] sepal_length score=1.25 DRIFTED | petal_length score=2.63 DRIFTED
[CIRCUIT] OK - 0 retrains today
[RETRAIN] Triggered: https://github.com/myorg/ml-pipeline/actions/runs/99999
Final: YELLOW | CI/CD loop closed!
```

Chat

LAB 6/6 860 PTS

MISSION TESTS HINTS DOCS

AUTOMATED CHECKS — MONITOR.PY

✓ Rolling accuracy computes drop from baseline

✓ Feature drift uses z-score (ref_std division)

✓ Circuit breaker checks max_retrains_per_day

✓ trigger_retraining sets triggered=True

✓ generate_alert returns structured dict

✓ ALL CHECKS PASSED

Lab complete. Next lab is now unlocked.

Git hub actions

AIM:

This project shows a simple end-to-end MLOps pipeline where a machine learning model is trained, validated, deployed, monitored, and retrained.

Project Flow

Train → Validate → CI Pipeline → Deploy → Monitor → Retrain

Project Structure

mlops-model-pipeline/ │ ├── train.py ├── validate.py ├── drift_detect.py ├── monitor.py ├── requirements.txt │ └── .github/ └── workflows/ ├── ci_pipeline.yml └── cd_deploy.yml

How to Run

Install dependencies: pip install -r requirements.txt
Run training: python train.py
Run validation: python validate.py
Run drift detection: python drift_detect.py
Run monitoring: python monitor.py

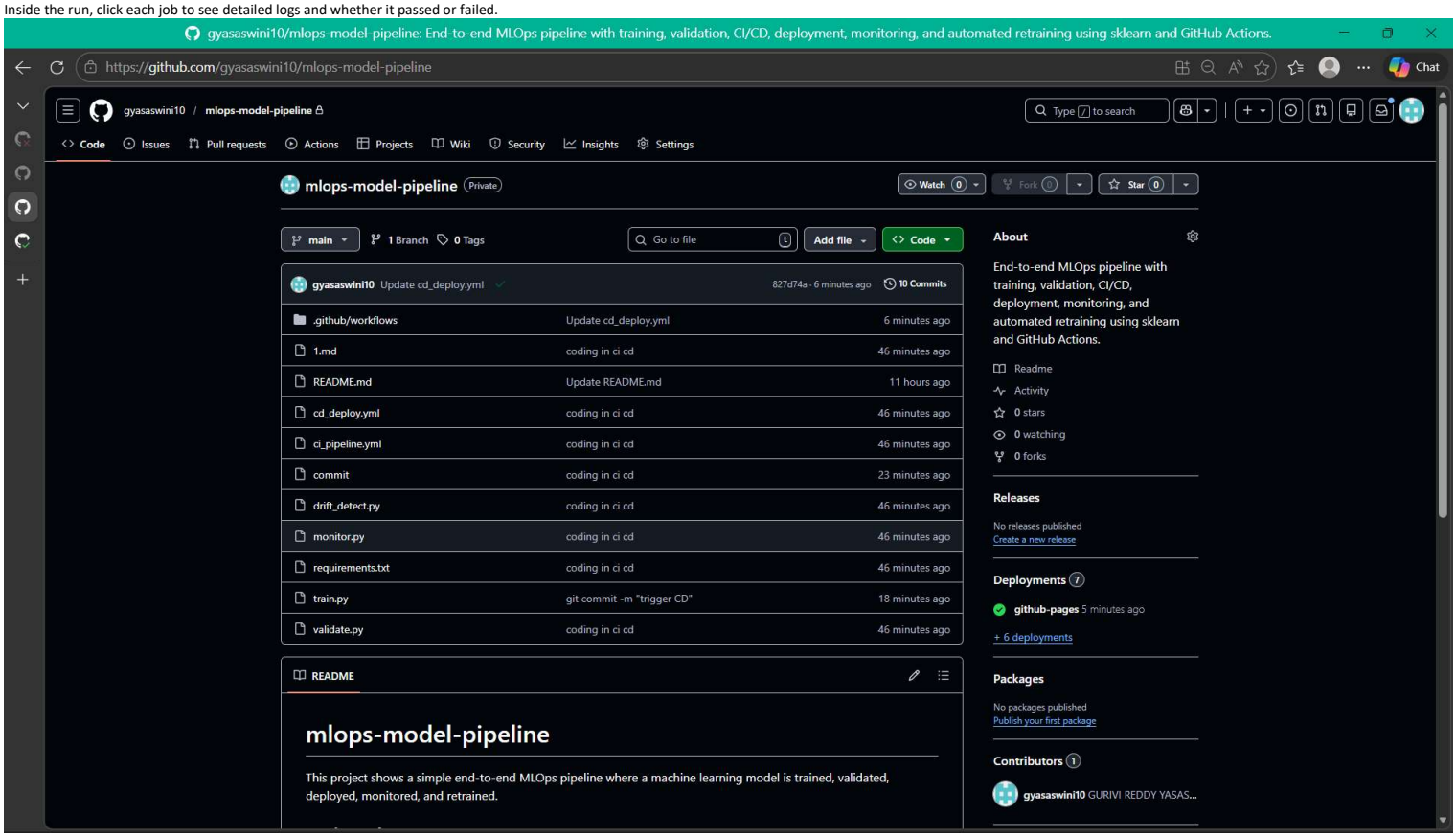
About

This project helps understand how ML models are used in real systems. Instead of training once, the model is checked, deployed carefully, monitored, and retrained when needed.

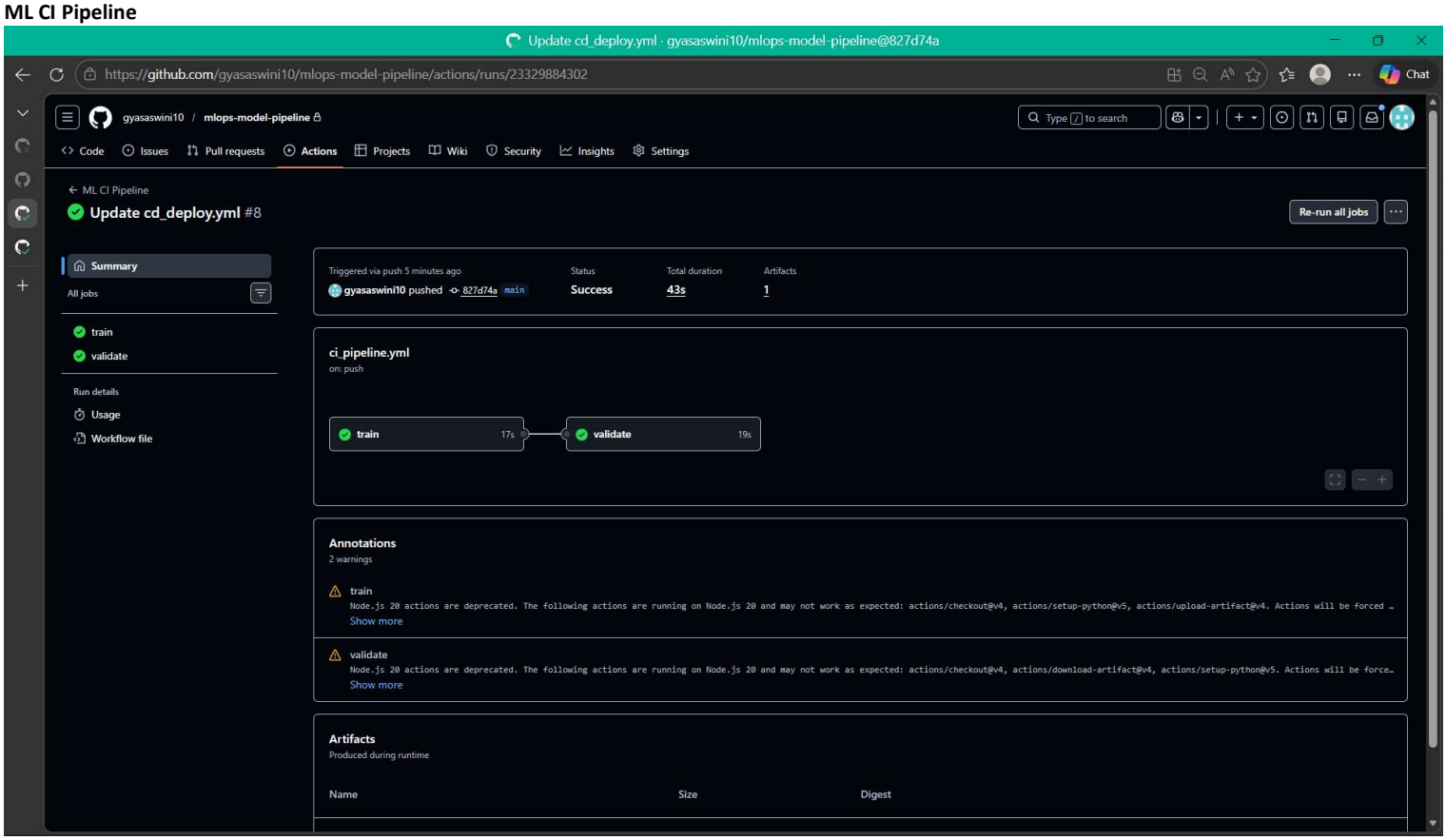
Link: https://github.com/gyasaswini10/mlops-model-pipeline.git

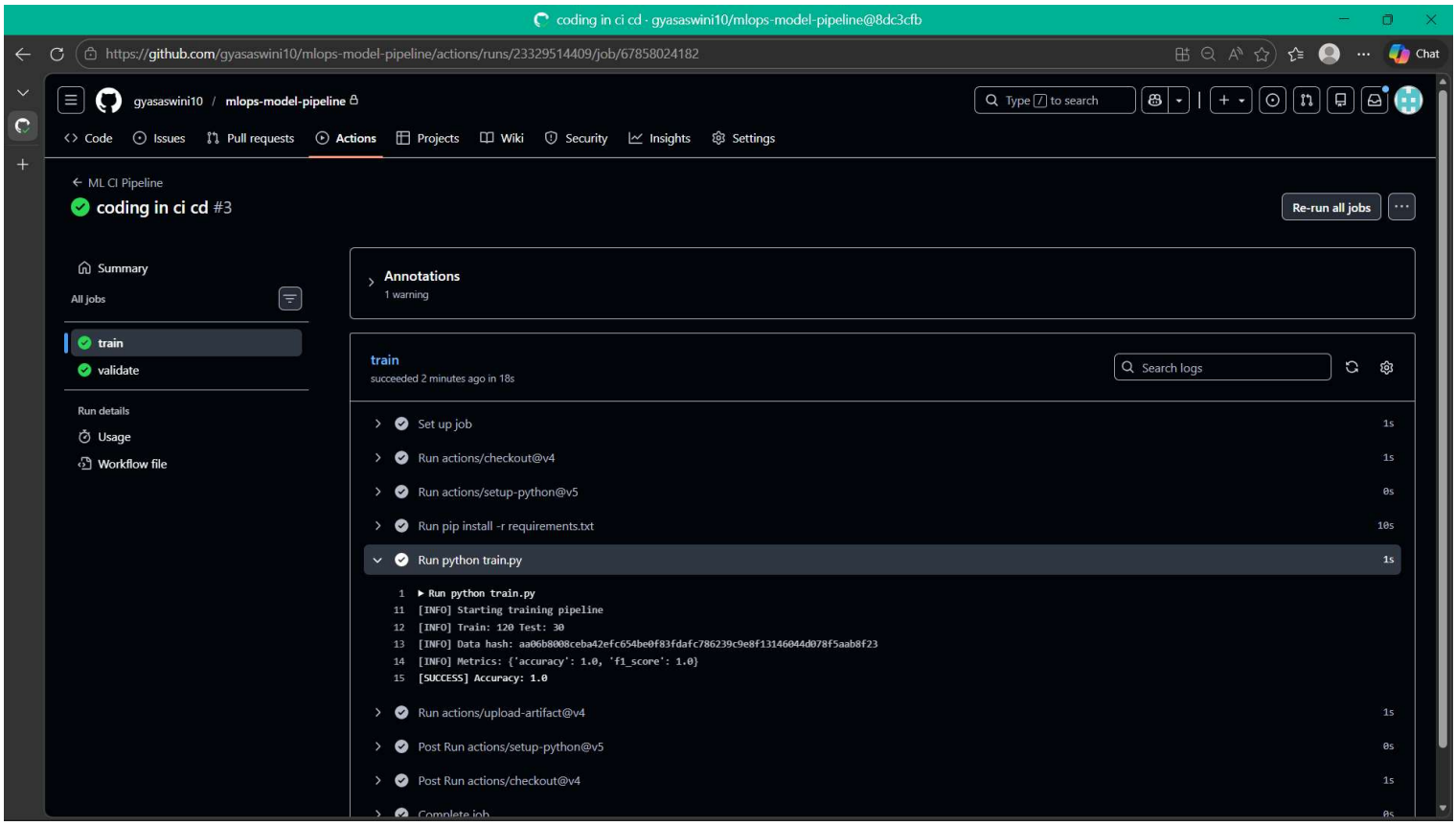
CI/CD Pipeline Verification:

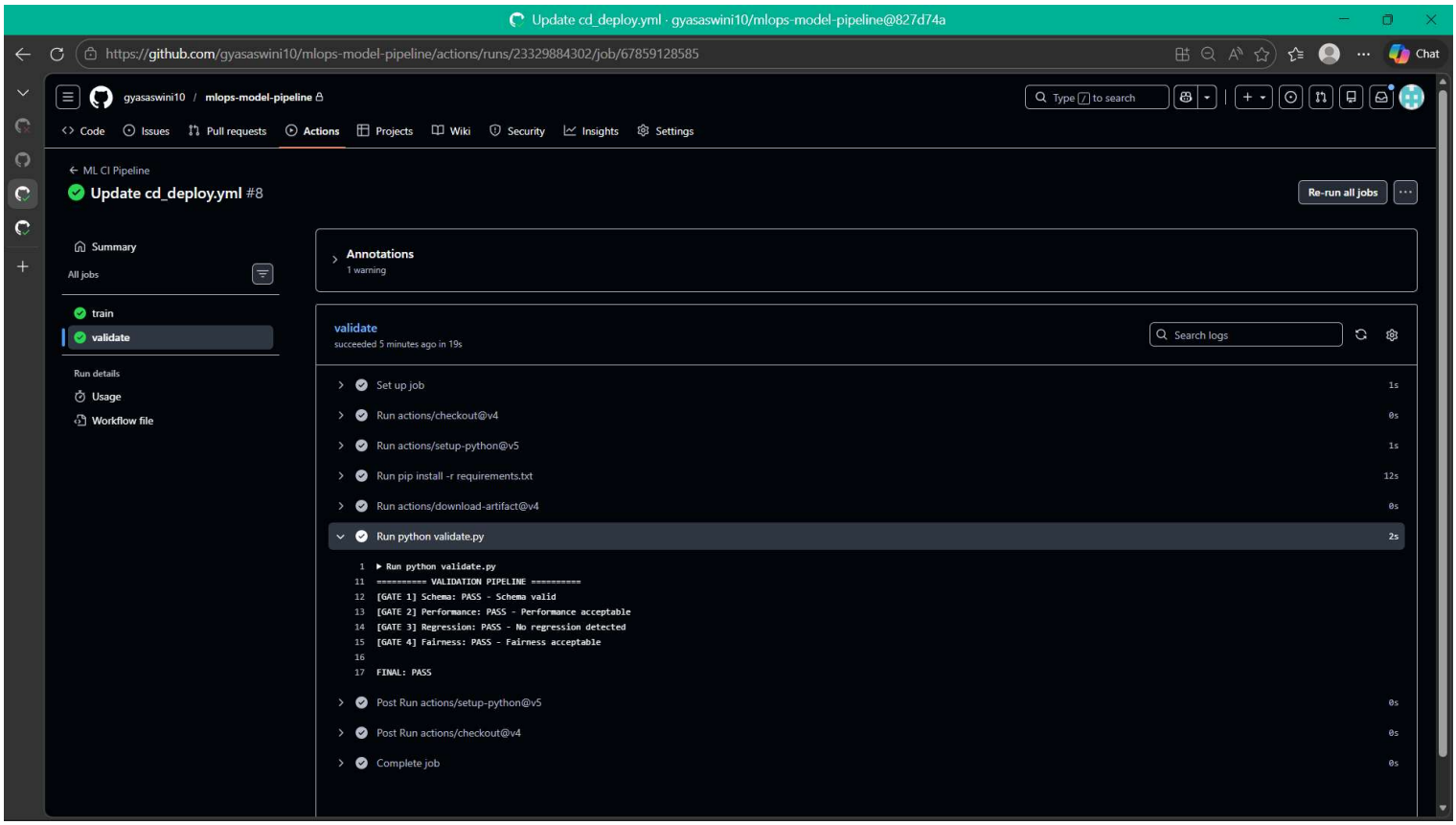
We can check my project in the GitHub repository under the **Actions tab**.
To see the CI/CD pipeline, click on **ML CI Pipeline** or **ML CD Pipeline** on the left side.
Then open any workflow run to view steps like training, validation, and deployment.
Inside the run, click each job to see detailed logs and whether it passed or failed.

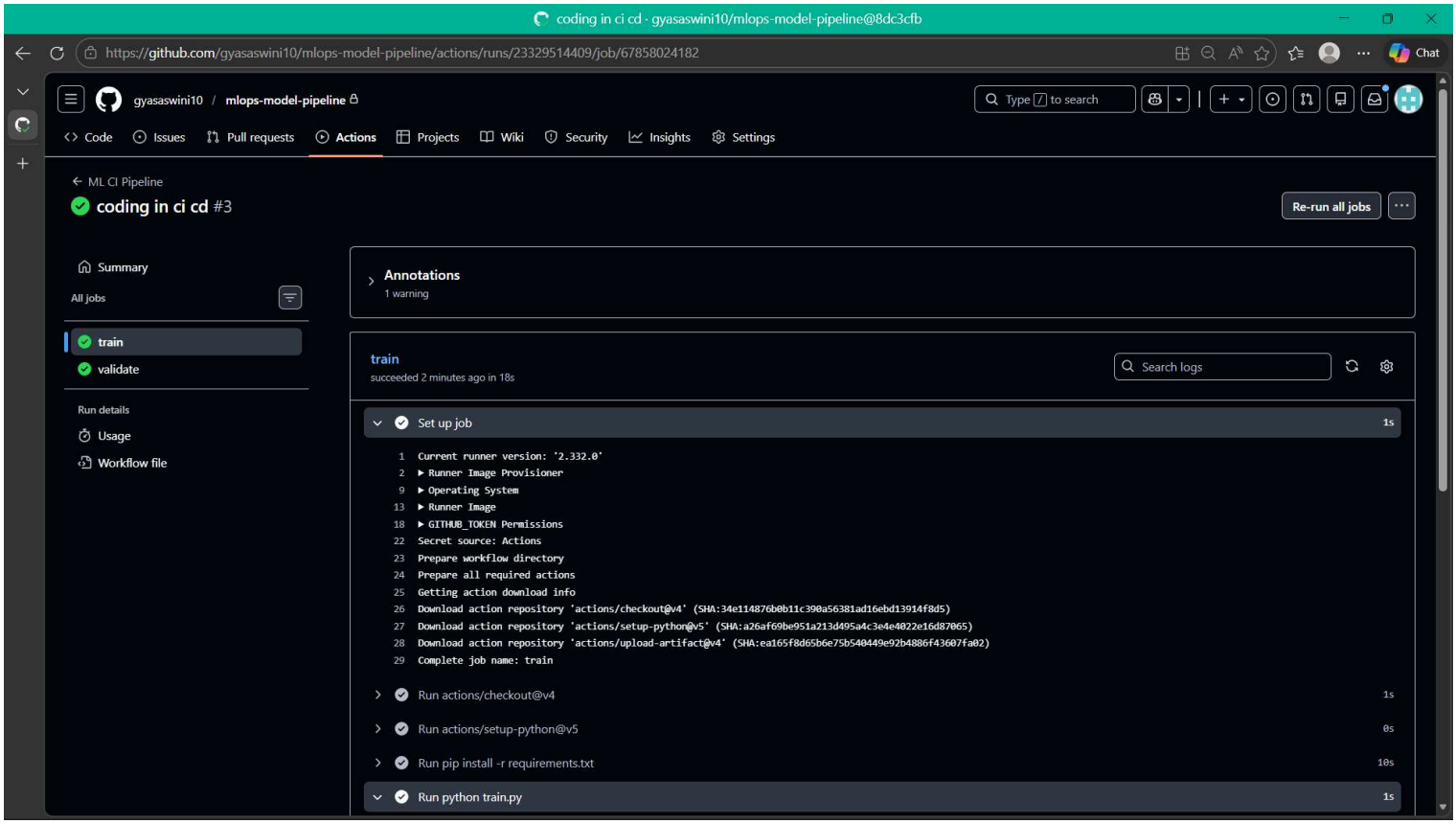


ML CI Pipeline

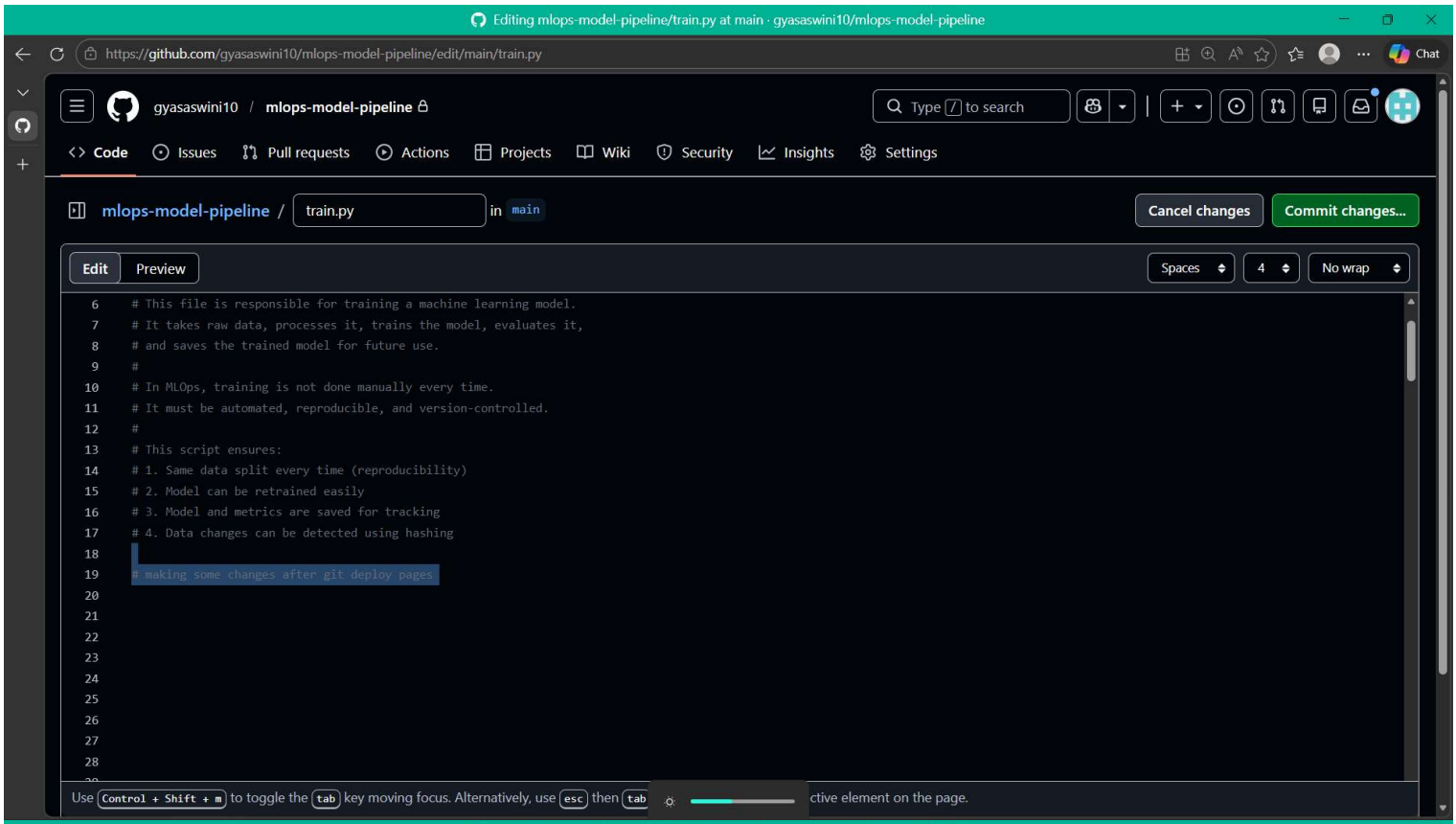


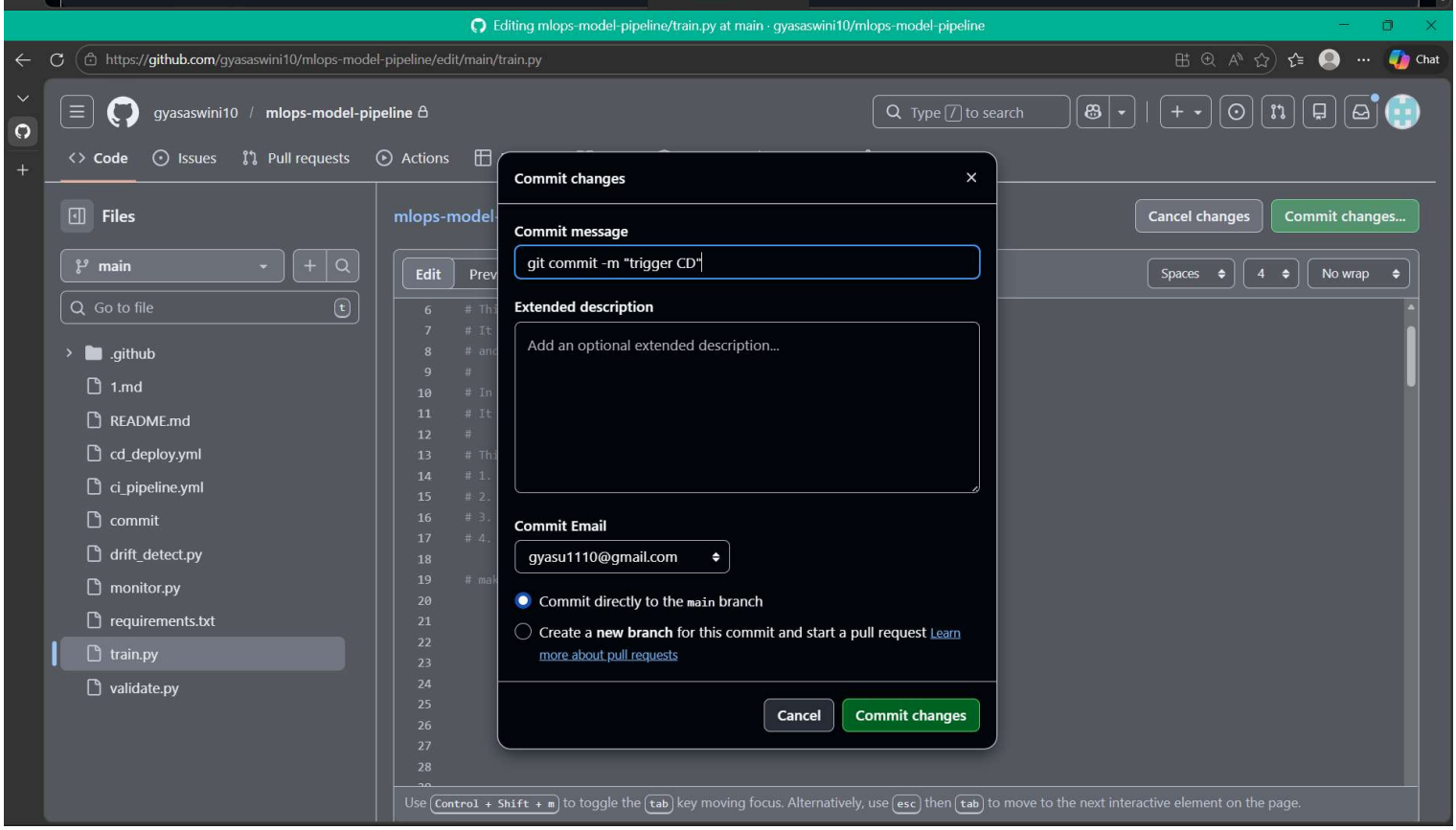


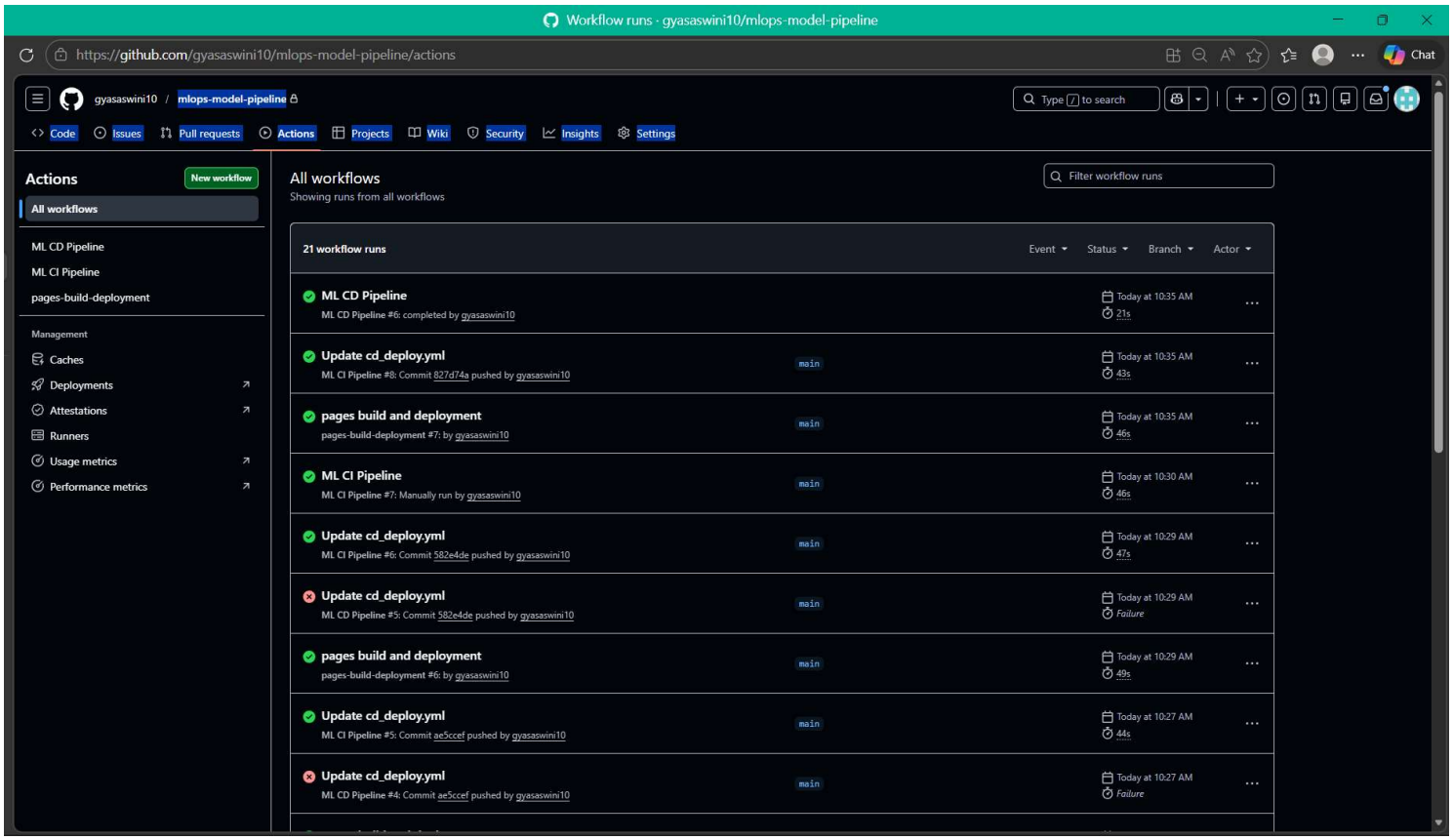


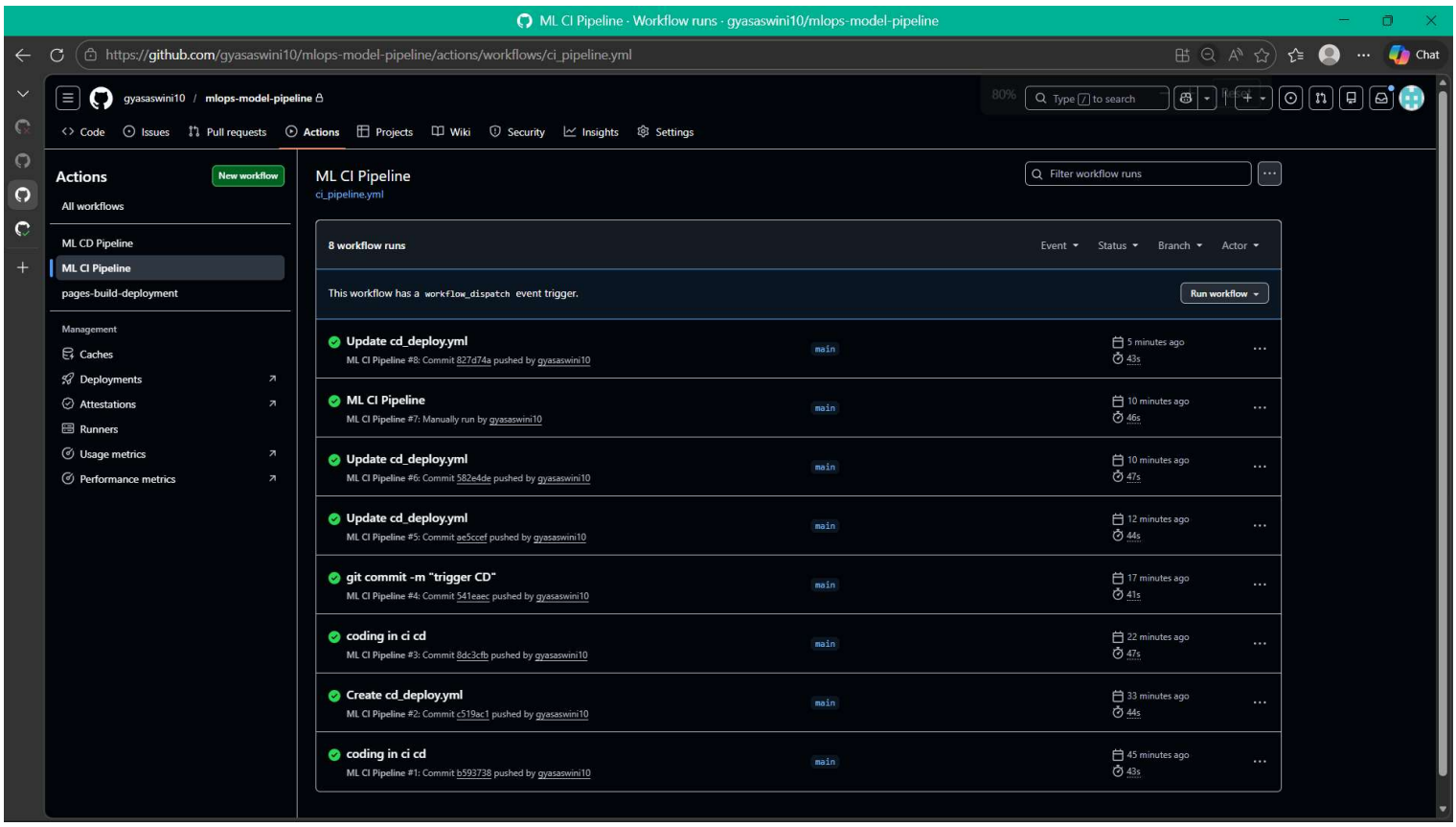


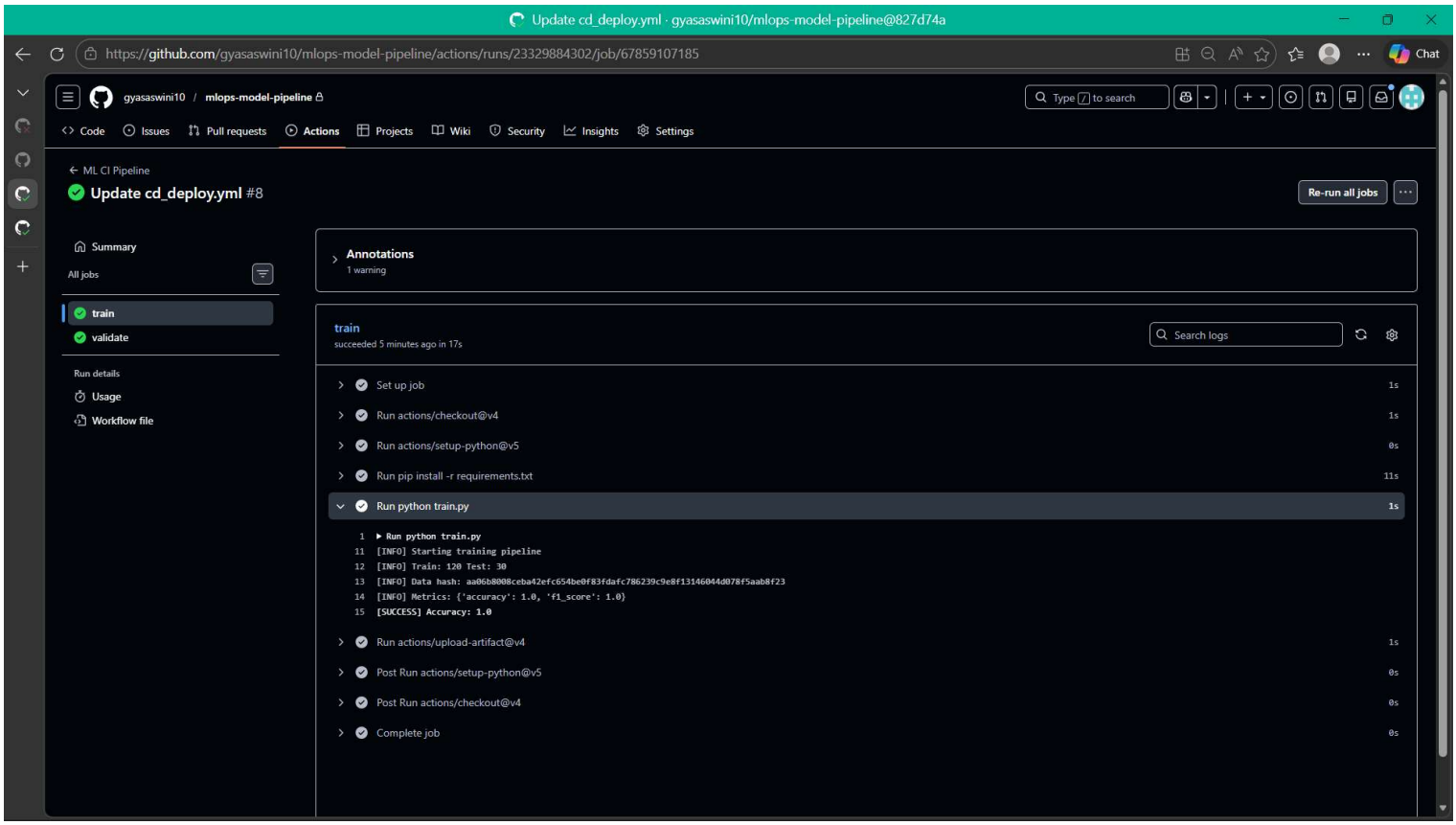

```
git add .
git commit -m "trigger CD"
git push
```


Editing mlops-model-pipeline/train.py at main · gyasaswini10/mlops-model-pipeline
https://github.com/gyasaswini10/mlops-model-pipeline/edit/main/train.py
gyasaswini10 / mlops-model-pipeline
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
mlops-model-pipeline / train.py in main
Cancel changes
Commit changes...
Edit
Preview
Spaces
4
No wrap
This file is responsible for training a machine learning model.
It takes raw data, processes it, trains the model, evaluates it,
and saves the trained model for future use.
#
In MLOps, training is not done manually every time.
It must be automated, reproducible, and version-controlled.
#
This script ensures:
1. Same data split every time (reproducibility)
2. Model can be retrained easily
3. Model and metrics are saved for tracking
4. Data changes can be detected using hashing
making some changes after git deploy pages
Use Control + Shift + m to toggle the tab key moving focus. Alternatively, use esc then tab


Editing mlops-model-pipeline/train.py at main · gyasaswini10/mlops-model-pipeline
https://github.com/gyasaswini10/mlops-model-pipeline/edit/main/train.py
gyasaswini10 / mlops-model-pipeline
Type / to search
Code
Issues
Pull requests
Actions
Files
main
Go to file
.github
1.md
README.md
cd_deploy.yml
ci_pipeline.yml
commit
drift_detect.py
monitor.py
requirements.txt
train.py
validate.py
Cancel changes
Commit changes...
Edit
Spaces
4
No wrap
Commit changes
Commit message
git commit -m "trigger CD"
Extended description
Add an optional extended description...
Commit Email
gyasu1110@gmail.com
Commit directly to the main branch
Create a new branch for this commit and start a pull request Learn more about pull requests
Cancel
Commit changes
Use Control + Shift + m to toggle the tab key moving focus. Alternatively, use esc then tab to move to the next interactive element on the page.


Workflow runs · gyasaswini10/mlops-model-pipeline
https://github.com/gyasaswini10/mlops-model-pipeline/actions
gyasaswini10 / mlops-model-pipeline
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
Actions
New workflow
All workflows
ML CD Pipeline
ML CI Pipeline
pages-build-deployment
Management
Caches
Deployments
Attestations
Runners
Usage metrics
Performance metrics
All workflows
Showing runs from all workflows
Filter workflow runs
21 workflow runs
Event
Status
Branch
Actor
ML CD Pipeline
ML CD Pipeline #6: completed by gyasaswini10
Today at 10:35 AM
21s
Update cd_deploy.yml
ML CI Pipeline #8: Commit 827d74a pushed by gyasaswini10
main
Today at 10:35 AM
43s
pages build and deployment
pages-build-deployment #7: by gyasaswini10
main
Today at 10:35 AM
46s
ML CI Pipeline
ML CI Pipeline #7: Manually run by gyasaswini10
main
Today at 10:30 AM
46s
Update cd_deploy.yml
ML CI Pipeline #6: Commit 582e4de pushed by gyasaswini10
main
Today at 10:29 AM
47s
Update cd_deploy.yml
ML CD Pipeline #5: Commit 582e4de pushed by gyasaswini10
main
Today at 10:29 AM
Failure
pages build and deployment
pages-build-deployment #6: by gyasaswini10
main
Today at 10:29 AM
49s
Update cd_deploy.yml
ML CI Pipeline #5: Commit ae5ccef pushed by gyasaswini10
main
Today at 10:27 AM
44s
Update cd_deploy.yml
ML CD Pipeline #4: Commit ae5ccef pushed by gyasaswini10
main
Today at 10:27 AM
Failure


ML CI Pipeline · Workflow runs · gyasaswini10/mlops-model-pipeline
https://github.com/gyasaswini10/mlops-model-pipeline/actions/workflows/ci_pipeline.yml
gyasaswini10 / mlops-model-pipeline
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
Actions
New workflow
All workflows
ML CD Pipeline
ML CI Pipeline
pages-build-deployment
Management
Caches
Deployments
Attestations
Runners
Usage metrics
Performance metrics
ML CI Pipeline
ci_pipeline.yml
Filter workflow runs
8 workflow runs
Event
Status
Branch
Actor
This workflow has a workflow_dispatch event trigger.
Run workflow
Update cd_deploy.yml
ML CI Pipeline #8: Commit 827d74a pushed by gyasaswini10
main
5 minutes ago
43s
ML CI Pipeline
ML CI Pipeline #7: Manually run by gyasaswini10
main
10 minutes ago
46s
Update cd_deploy.yml
ML CI Pipeline #6: Commit 582e4de pushed by gyasaswini10
main
10 minutes ago
47s
Update cd_deploy.yml
ML CI Pipeline #5: Commit ae5ccef pushed by gyasaswini10
main
12 minutes ago
44s
git commit -m "trigger CD"
ML CI Pipeline #4: Commit 541eaec pushed by gyasaswini10
main
17 minutes ago
41s
coding in ci cd
ML CI Pipeline #3: Commit 8dc3cfb pushed by gyasaswini10
main
22 minutes ago
47s
Create cd_deploy.yml
ML CI Pipeline #2: Commit c519ac1 pushed by gyasaswini10
main
33 minutes ago
44s
coding in ci cd
ML CI Pipeline #1: Commit b593738 pushed by gyasaswini10
main
45 minutes ago
43s



ML CD Pipeline

Issues:

CI/CD YAML mistakes – At first, my CD pipeline didn't run because the workflow name in cd_deploy.yml didn't match the CI pipeline name. I fixed it by renaming the CD workflow to ML CD Deploy and making sure workflows: ["ML CI Pipeline"] exactly matched the CI workflow name.
Duplicate keys in YAML – I accidentally had workflows written twice. YAML doesn't allow that, so I removed the duplicate to make the file valid.
What to replace:

- Replace the name: in cd_deploy.yml from:

name: ML CI Pipeline
to:

name: ML CD Deploy

- Make sure the trigger line matches the CI workflow name exactly:

workflows: ["ML CI Pipeline"]
Why:
If the CD workflow has the same name as CI, it tries to trigger itself → GitHub Actions fails. Also, the workflow trigger must match the **CI pipeline's exact name** to run after it completes successfully.

gyasaswini10 / mlops-model-pipeline

Code | Issues | Pull requests | Actions | Projects | Wiki | Security | Insights | Settings

Type / to search

Actions New workflow

All workflows

ML CD Pipeline

ML CI Pipeline

pages-build-deployment

Management

- Caches
- Deployments
- Attestations
- Runners
- Usage metrics
- Performance metrics

ML CD Pipeline

cd_deploy.yml

Filter workflow runs

6 workflow runs Event | Status | Branch | Actor

ML CD Pipeline ML CD Pipeline #6: completed by gyasaswini10		4 minutes ago 21s	
Update cd_deploy.yml ML CD Pipeline #5: Commit 582e4de pushed by gyasaswini10	main	9 minutes ago Failure	
Update cd_deploy.yml ML CD Pipeline #4: Commit ae5ccef pushed by gyasaswini10	main	12 minutes ago Failure	
git commit -m "trigger CD" ML CD Pipeline #3: Commit 541eaec pushed by gyasaswini10	main	16 minutes ago Failure	
coding in ci cd ML CD Pipeline #2: Commit 8dc3cfb pushed by gyasaswini10	main	21 minutes ago Failure	
Create cd_deploy.yml ML CD Pipeline #1: Commit c519ac1 pushed by gyasaswini10	main	33 minutes ago Failure	

← ML CD Pipeline

ML CD Pipeline #6

Re-run all jobs

Summary

All jobs

run_cd

Run details

- Usage
- Workflow file

Annotations
1 warning

run_cd
succeeded 3 minutes ago in 16s

Search logs

- Set up job 1s
- Checkout Code 1s
- Setup Python 0s
- Install Dependencies 10s
- Run Monitoring 1s

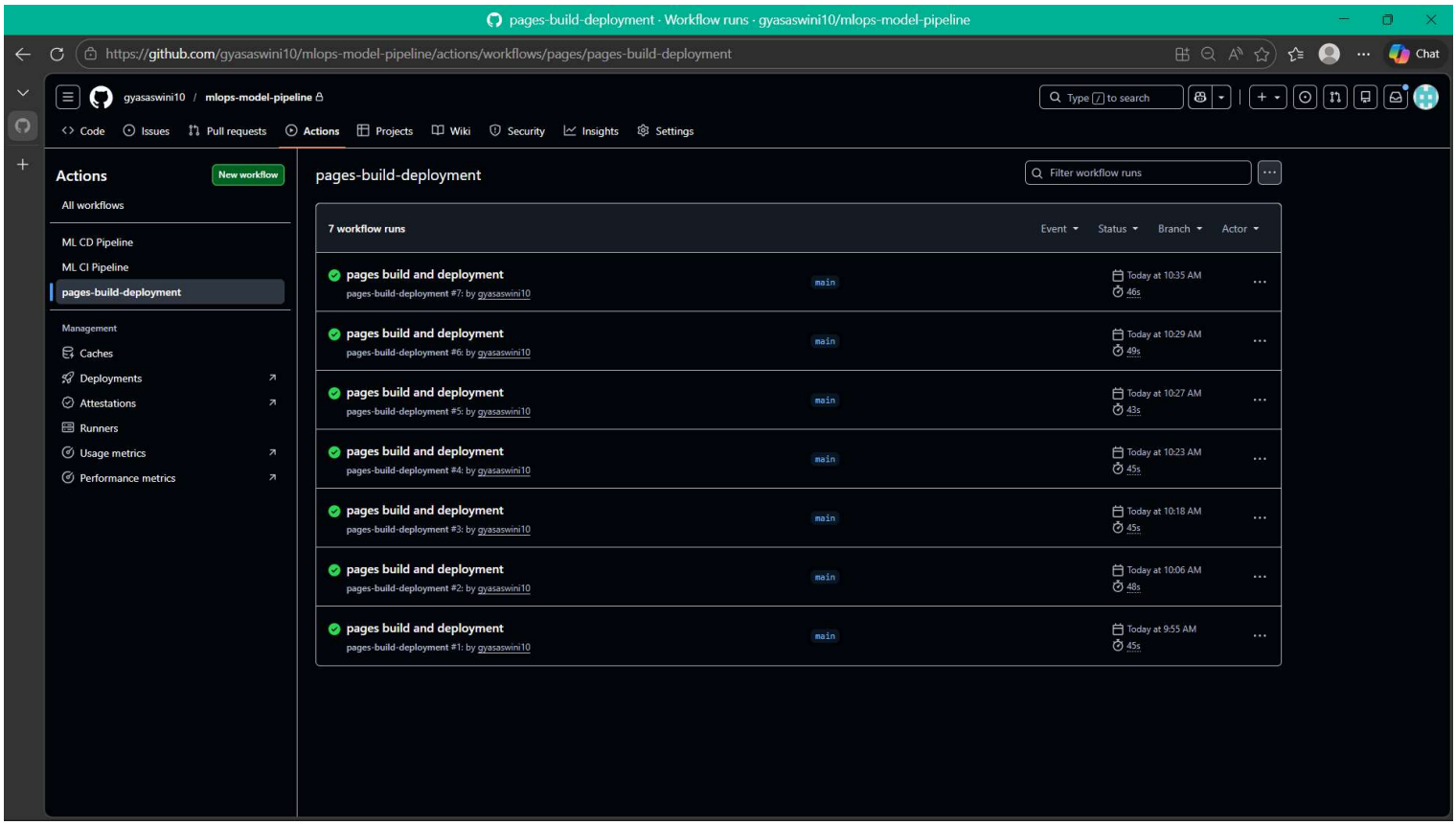
```
▶ Run python monitor.py
MONITORING - 2026-03-20T05:06:11.040350
[ACCURACY] 0.8 | drop=0.08
[DRIFT] sepal_length → 0.6
[DRIFT] sepal_width → 0.25
[DRIFT] petal_length → 0.67
[DRIFT] petal_width → 0.25

FINAL STATUS: YELLOW
```

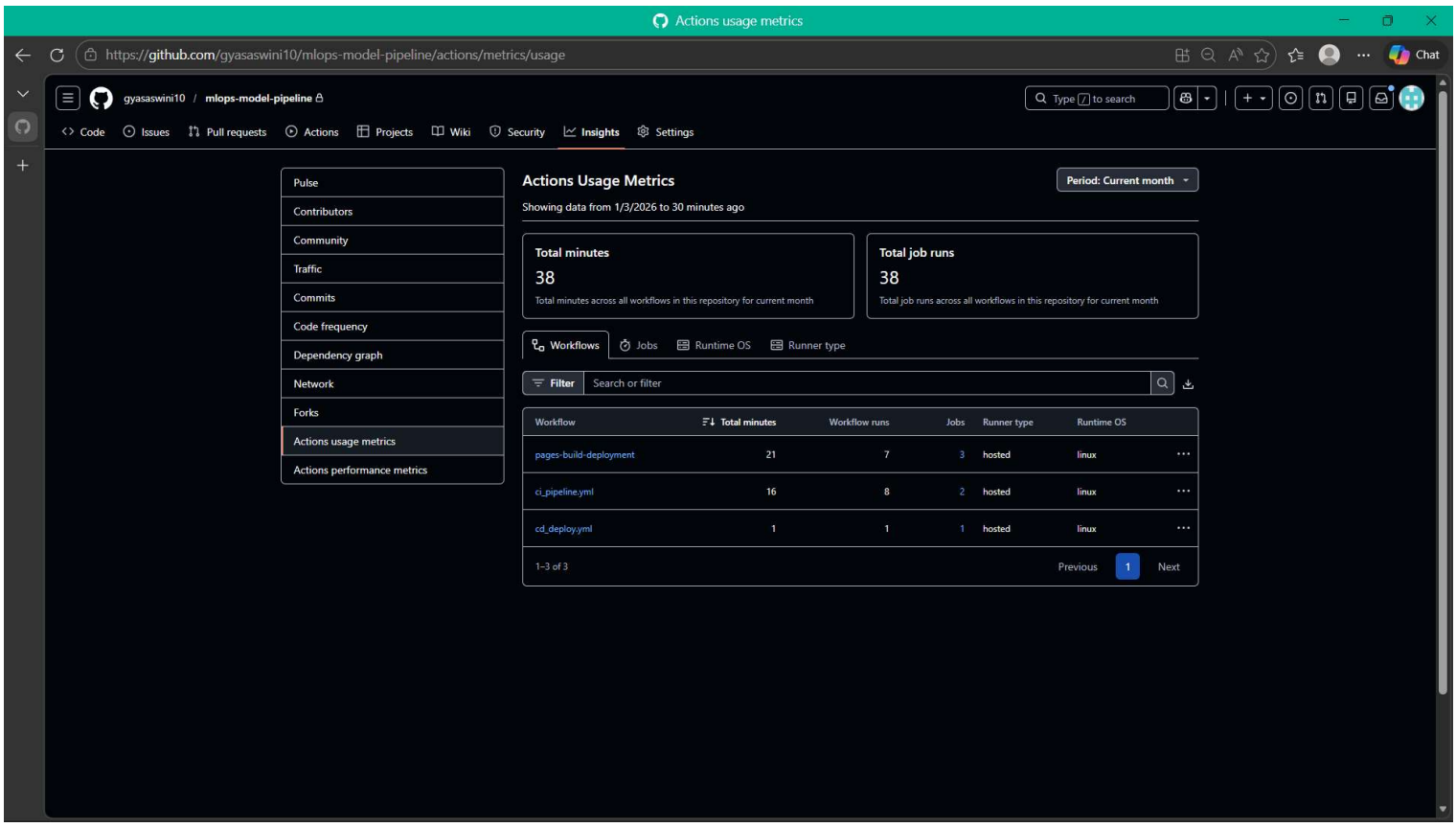
- Run Drift Detection 1s

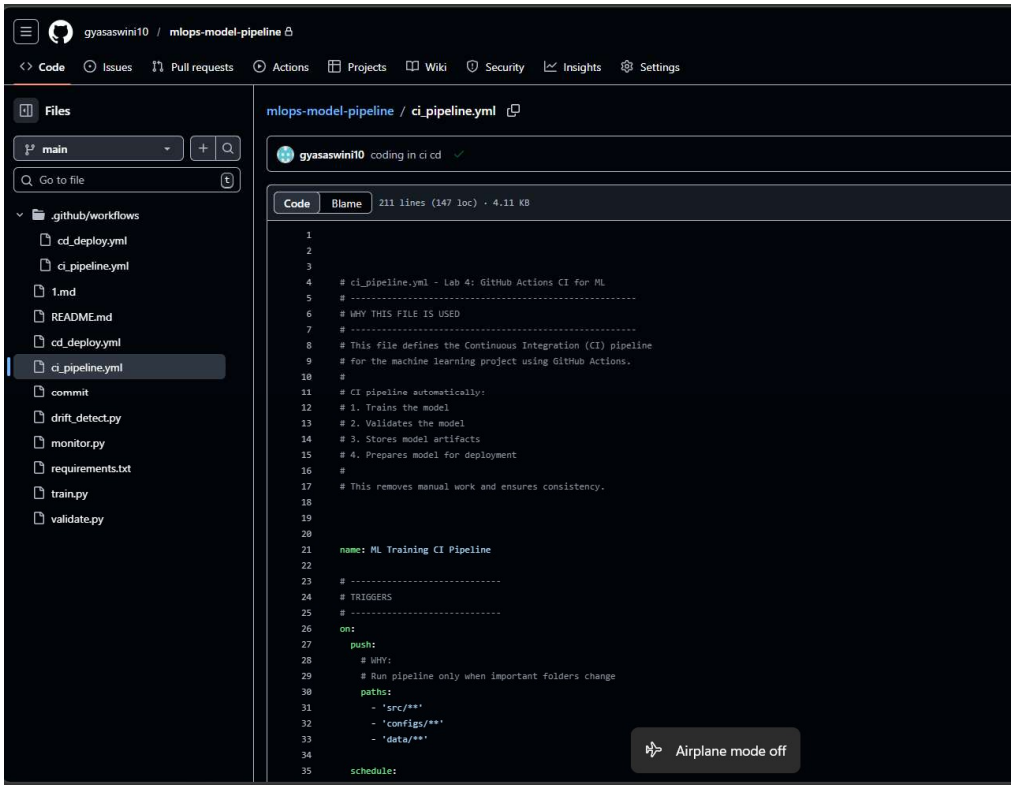
```
▶ Run python drift_detect.py
========== DRIFT DETECTION ==========
sepal_length → PSI=2.4754 | KL=0.375 | severe
sepal_width → PSI=0.173 | KL=0.0686 | slight
petal_length → PSI=4.2984 | KL=2.4598 | severe
petal_width → PSI=2.3866 | KL=0.3979 | severe

OVERALL STATUS: RED
RECOMMENDATION: Immediate retraining required
```

- Done 0s

Airplane mode off

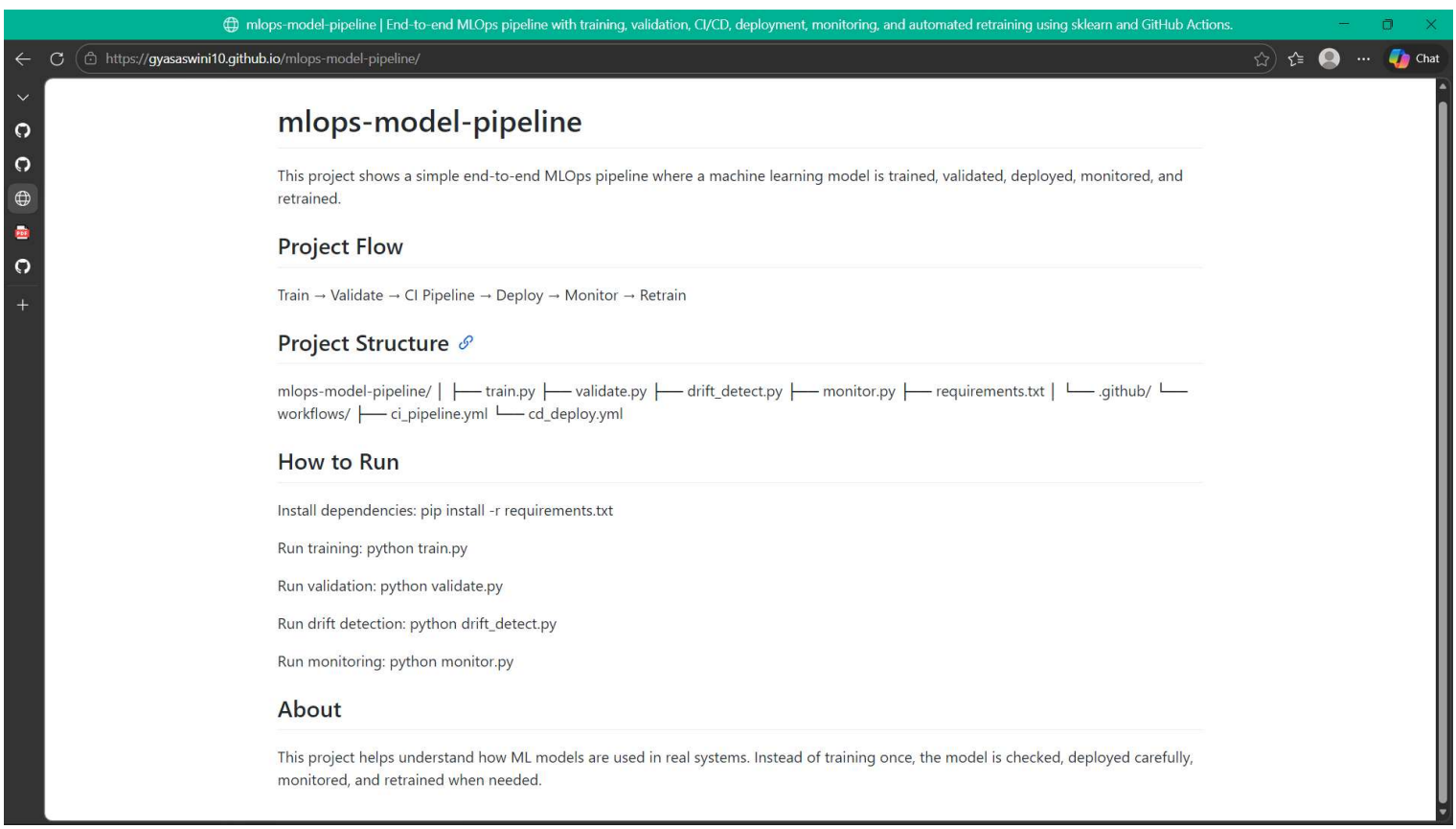

pages-build-deployment · Workflow runs · gyasaswini10/mlops-model-pipeline
https://github.com/gyasaswini10/mlops-model-pipeline/actions/workflows/pages/pages-build-deployment
gyasaswini10 / mlops-model-pipeline
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
Actions
New workflow
All workflows
ML CD Pipeline
ML CI Pipeline
pages-build-deployment
Management
Caches
Deployments
Attestations
Runners
Usage metrics
Performance metrics
pages-build-deployment
Filter workflow runs
7 workflow runs
Event
Status
Branch
Actor
pages build and deployment
pages-build-deployment #7: by gyasaswini10
main
Today at 10:35 AM
46s
pages build and deployment
pages-build-deployment #6: by gyasaswini10
main
Today at 10:29 AM
49s
pages build and deployment
pages-build-deployment #5: by gyasaswini10
main
Today at 10:27 AM
43s
pages build and deployment
pages-build-deployment #4: by gyasaswini10
main
Today at 10:23 AM
45s
pages build and deployment
pages-build-deployment #3: by gyasaswini10
main
Today at 10:18 AM
45s
pages build and deployment
pages-build-deployment #2: by gyasaswini10
main
Today at 10:06 AM
48s
pages build and deployment
pages-build-deployment #1: by gyasaswini10
main
Today at 9:55 AM
45s


Actions usage metrics
https://github.com/gyasaswini10/mlops-model-pipeline/actions/metrics/usage
gyasaswini10 / mlops-model-pipeline
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
Pulse
Contributors
Community
Traffic
Commits
Code frequency
Dependency graph
Network
Forks
Actions usage metrics
Actions performance metrics
Actions Usage Metrics
Period: Current month
Showing data from 1/3/2026 to 30 minutes ago
Total minutes
38
Total minutes across all workflows in this repository for current month
Total job runs
38
Total job runs across all workflows in this repository for current month
Workflows
Jobs
Runtime OS
Runner type
Filter
Search or filter
Workflow
Total minutes
Workflow runs
Jobs
Runner type
Runtime OS
pages-build-deployment 21 7 3 hosted linux
ci_pipeline.yml 16 8 2 hosted linux
cd_deploy.yml 1 1 1 hosted linux
1–3 of 3
Previous
1
Next



LINK: Thi s page is GitHub Pages, and it is displaying my **README.md file as a website**.
It is used to present my project details like flow, structure, and steps.
It is not running the model or pipeline.
Actual execution (CI/CD) happens in the **Actions tab**

Link: https://gyasaswini10.github.io/mlops-model-pipeline/



OBSERVATION:
I worked on an MLOps pipeline project using the Iris dataset. I first trained a Random Forest model and split the data into training and testing sets to check how well it performed. I saved the trained model and metadata to keep track of changes. After training, I ran validation checks to make sure the model was accurate, fair, and didn't regress compared to previous versions. I also tested for data drift to see if production data was different from training data. Monitoring was set up to track accuracy and latency while the model ran in

production. Setting up the CI/CD pipeline was challenging because small YAML errors kept breaking the workflow, but I successfully implemented GitHub Actions to automate training, validation, and deployment. Every stage felt important—training created the model, validation ensured quality, CI/CD automated the process, deployment delivered the model safely, and monitoring caught issues early. Overall, this project helped me understand how machine learning works in real systems, not just in notebooks.